TIME SERIES FORECASTING - ROSE WINE DATASET

Problem Statement:

For this particular assignment, the data of different types of wine sales in the 20th century is to be analysed. Both of these data are from the same company but of different wines. As an analyst in the ABC Estate Wines, you are tasked to analyse and forecast Wine Sales in the 20th century.

In [1]:
```
#First importing all the important libraries and packages

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from pylab import rcParams

import statsmodels.api as sm
from warnings import filterwarnings
filterwarnings('ignore') #to ignore all kind of warnings including future warnings
```

1. Read the data as an appropriate Time Series data and plot the data.

In [2]:
```
rose_data = pd.read_csv(r'D:\SHUBHANK !\GL\8. TOPIC 7 - Time Series Forecasting\Final p
```

In [3]:
```
#Let's check head

rose_data.head()
```

Out[3]:

	YearMonth	Rose
0	1980-01	112.0
1	1980-02	118.0
2	1980-03	129.0
3	1980-04	99.0
4	1980-05	116.0

In [4]:
```
rose_data.tail()
```

Out[4]:

	YearMonth	Rose
182	1995-03	45.0
183	1995-04	52.0
184	1995-05	28.0

	YearMonth	Rose
185	1995-06	40.0
186	1995-07	62.0

In [5]:
```
#Summary/Description of the data

rose_data.describe()
```

Out[5]:

	Rose
count	185.000000
mean	90.394595
std	39.175344
min	28.000000
25%	63.000000
50%	86.000000
75%	112.000000
max	267.000000

In [6]:
```
#Info of the data

rose_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 187 entries, 0 to 186
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   YearMonth  187 non-null    object 
 1   Rose       185 non-null    float64
dtypes: float64(1), object(1)
memory usage: 3.0+ KB
```

In [7]:
```
#shape of the data

rose_data.shape
```

Out[7]:
```
(187, 2)
```

In [8]:
```
#checking null values

rose_data.isnull().any()
```

Out[8]:
```
YearMonth    False
Rose          True
dtype: bool
```

In [9]:

```
#null values are present in the 'Rose' column

#Let's check how many null values are present

rose_data.isna().sum()
```

Out[9]:
```
YearMonth    0
Rose         2
dtype: int64
```

2 missing values are present

In [10]:
```
# Let's make it a time series dataframe with a monthly frequency

full_date = pd.date_range(start = '01/01/1980', end = '08/01/1995', freq = 'M')

full_date
```

Out[10]:
```
DatetimeIndex(['1980-01-31', '1980-02-29', '1980-03-31', '1980-04-30',
               '1980-05-31', '1980-06-30', '1980-07-31', '1980-08-31',
               '1980-09-30', '1980-10-31',
               ...
               '1994-10-31', '1994-11-30', '1994-12-31', '1995-01-31',
               '1995-02-28', '1995-03-31', '1995-04-30', '1995-05-31',
               '1995-06-30', '1995-07-31'],
              dtype='datetime64[ns]', length=187, freq='M')
```

In [11]:
```
#Now, we will add it into the dataframe with the column name 'Time_stamp'

rose_data['Time_stamp'] = pd.DataFrame(full_date)

#and set this column as index

rose_data.set_index('Time_stamp', inplace = True)

#Also then, we will drop the 'YearMonth' column from the dataset

rose_data.drop('YearMonth', axis = 1 , inplace = True)
```

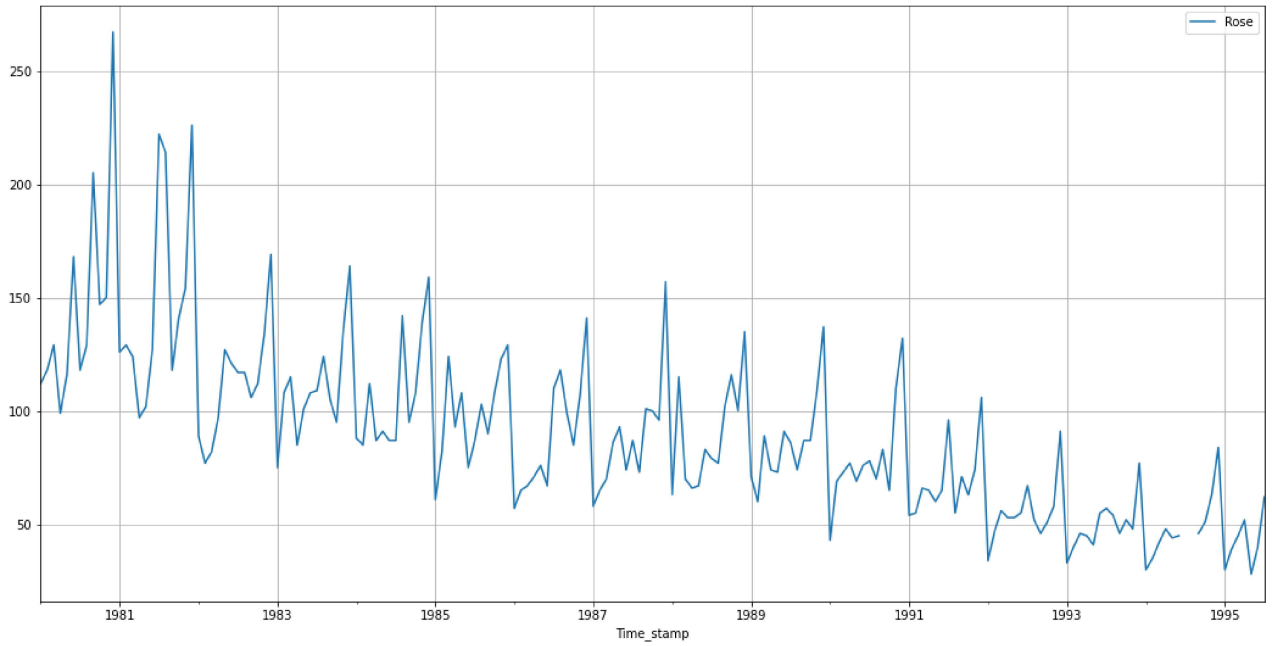
In [12]:
```
#Let's check the rose_data again
rose_data.head()
```

Out[12]:

Time_stamp	Rose
1980-01-31	112.0
1980-02-29	118.0
1980-03-31	129.0
1980-04-30	99.0
1980-05-31	116.0

In [13]:
```
#Plot the time series to understand the behaviour of data
```

```
rcParams['figure.figsize'] = 18,9

rose_data.plot()
plt.grid()
plt.show()
```

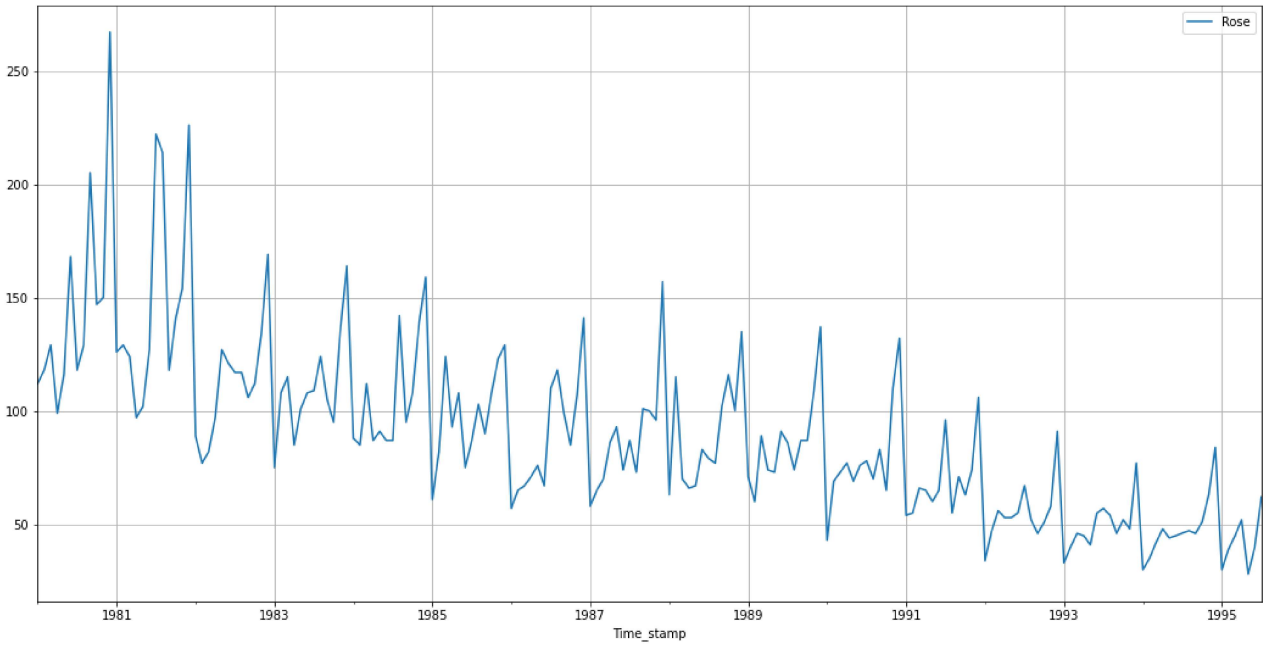


It is clearly visible that dataset has decreasing seasonality and trend. Also we can see that the year 1994 has the missing data

Now first we have to treat the missing observations in the dataset

We are using interpolation method to treat the missing observations. This interpolation method is to use a spline to connect the values. Using a spline interpolation requires you specify the order (number of terms in the polynomial). In our case, an order of 1 is just fine.

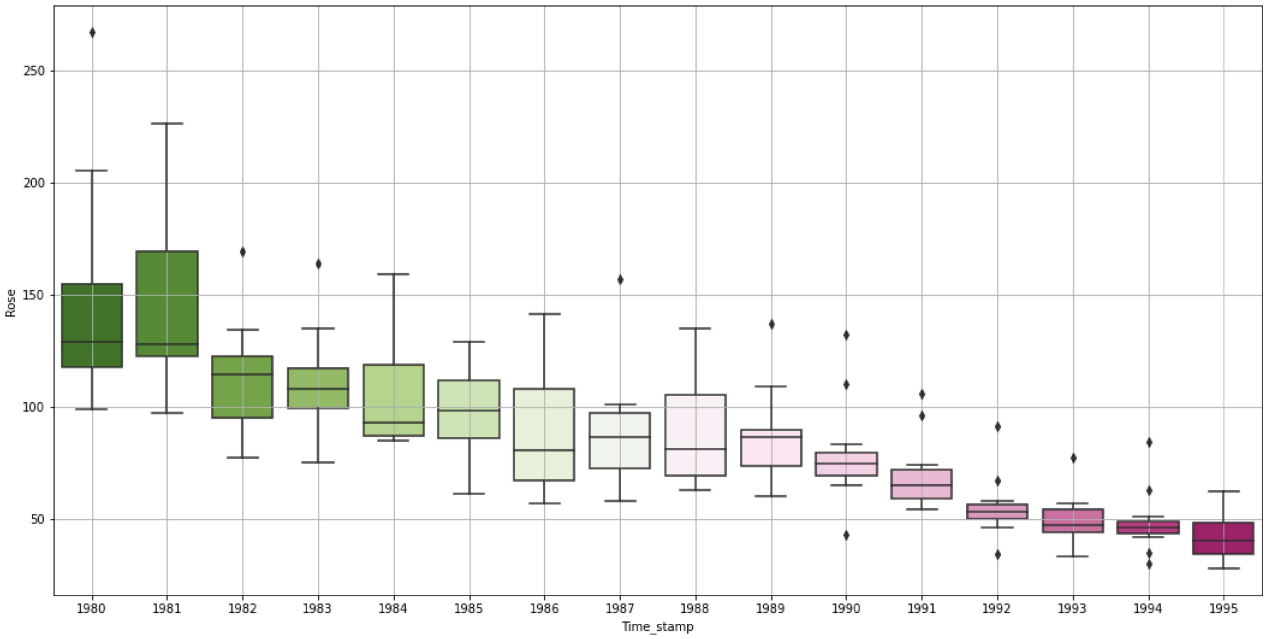
In [14]:
```
rose_data = rose_data.interpolate(method='spline',order=1)

#re-plotting

rose_data.plot()
plt.grid()
plt.show()
```



Now we can see there are no missing points observed in the above plot.

2. Perform appropriate Exploratory Data Analysis to understand the data and also perform decomposition.

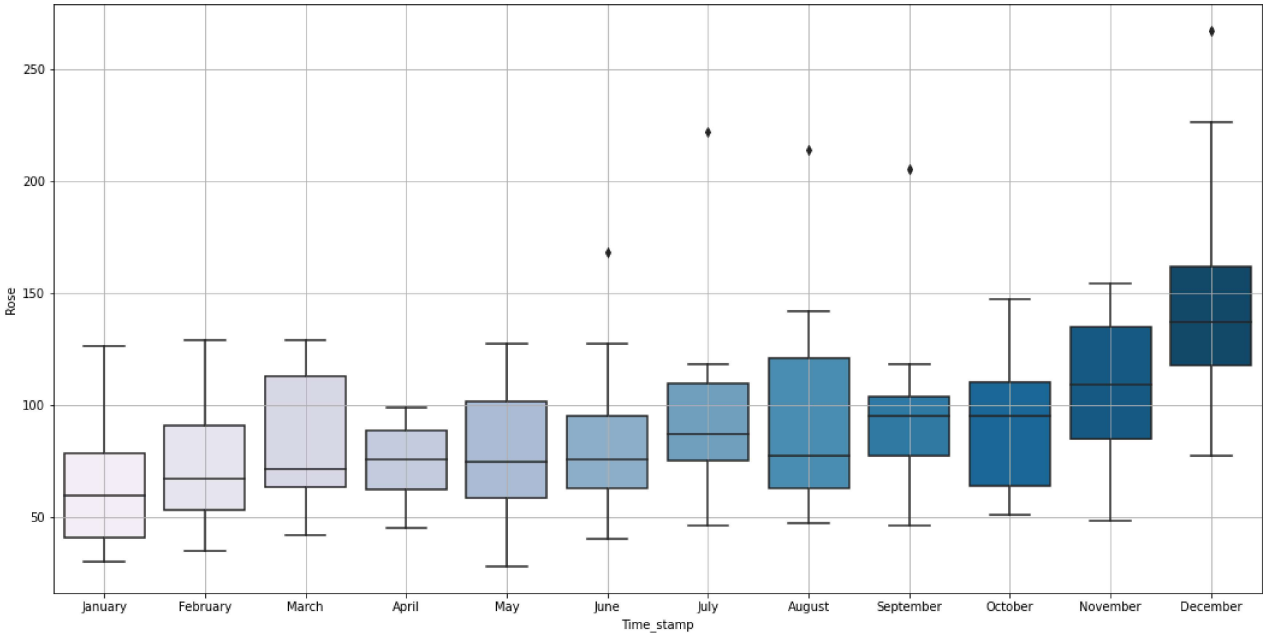
In [15]:
```
#Building a yearly boxplot

sns.boxplot(x = rose_data.index.year, y = rose_data['Rose'], palette='PiYG_r')

plt.grid()
plt.show()
```



Fall in trend is clearly visible from yearly plot. Now let's see monthly plot.

In [16]:
```
#now let's look at the monthly boxplot
```

```
sns.boxplot(x = rose_data.index.month_name(), y = rose_data['Rose'], palette='PuBu')

plt.grid()
plt.show()
```



In monthly plot, the trend is growing.

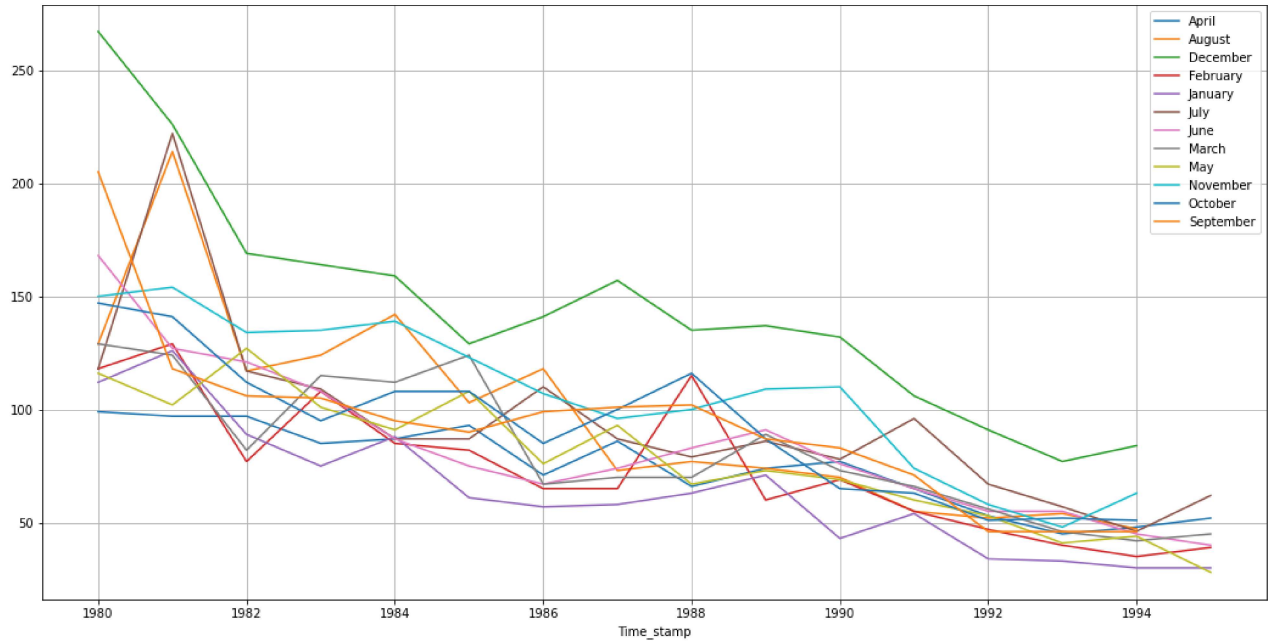
In [17]:
```
#Also by using pivot table with respect to sales, we can get some insights

sales_monthwise_per_year = pd.pivot_table(rose_data, values = 'Rose',
                                columns = rose_data.index.month_name(),
                                index = rose_data.index.year)

sales_monthwise_per_year
```

Out[17]:

Time_stamp	April	August	December	February	January	July	June	March	May	Novem
Time_stamp										
1980	99.0	129.000000	267.0	118.0	112.0	118.000000	168.0	129.0	116.0	15
1981	97.0	214.000000	226.0	129.0	126.0	222.000000	127.0	124.0	102.0	15
1982	97.0	117.000000	169.0	77.0	89.0	117.000000	121.0	82.0	127.0	13
1983	85.0	124.000000	164.0	108.0	75.0	109.000000	108.0	115.0	101.0	13
1984	87.0	142.000000	159.0	85.0	88.0	87.000000	87.0	112.0	91.0	13
1985	93.0	103.000000	129.0	82.0	61.0	87.000000	75.0	124.0	108.0	12
1986	71.0	118.000000	141.0	65.0	57.0	110.000000	67.0	67.0	76.0	10
1987	86.0	73.000000	157.0	65.0	58.0	87.000000	74.0	70.0	93.0	9
1988	66.0	77.000000	135.0	115.0	63.0	79.000000	83.0	70.0	67.0	10
1989	74.0	74.000000	137.0	60.0	71.0	86.000000	91.0	89.0	73.0	10
1990	77.0	70.000000	132.0	69.0	43.0	78.000000	76.0	73.0	69.0	11

Time_stamp	April	August	December	February	January	July	June	March	May	Novem
Time_stamp										
1991	65.0	55.000000	106.0	55.0	54.0	96.000000	65.0	66.0	60.0	7
1992	53.0	52.000000	91.0	47.0	34.0	67.000000	55.0	56.0	53.0	5
1993	45.0	54.000000	77.0	40.0	33.0	57.000000	55.0	46.0	41.0	4
1994	48.0	47.221907	84.0	35.0	30.0	46.155493	45.0	42.0	44.0	6
1995	52.0	NaN	NaN	39.0	30.0	62.000000	40.0	45.0	28.0	N

In [18]:
```
#Now let's look at the plot of the above data monthwise

sales_monthwise_per_year.plot()

plt.grid()
plt.legend(loc='best')
plt.show()
```

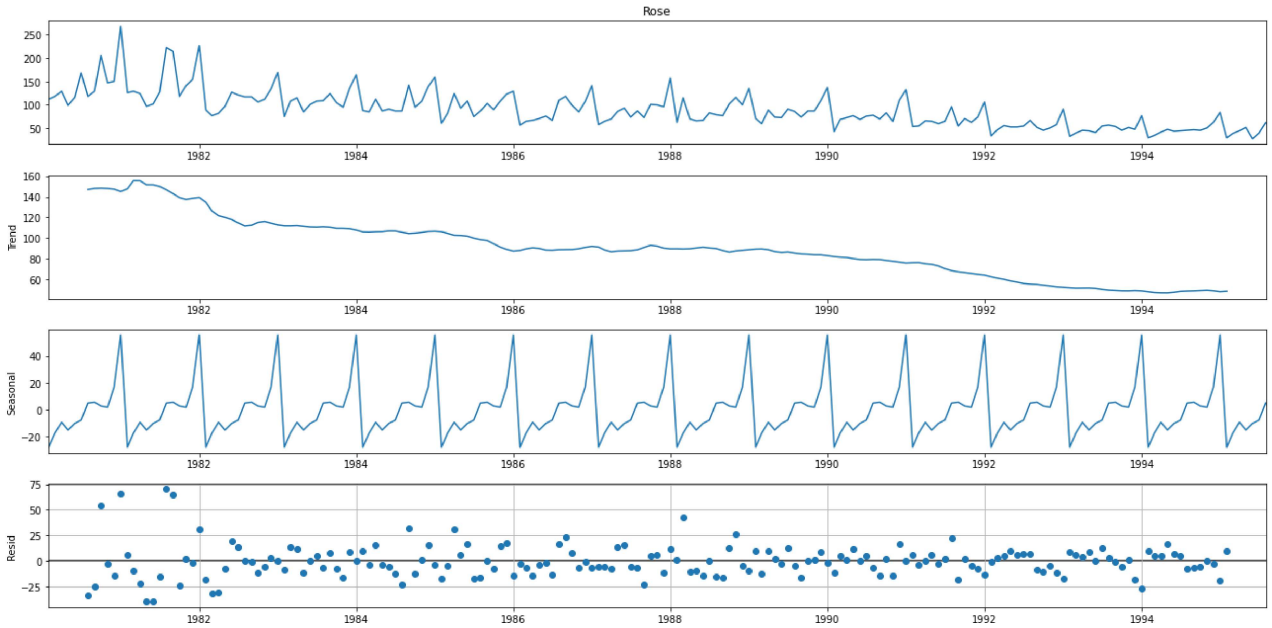


From the above plot, every year the sales in December month(represented by green line) are higher than other months.

In [19]:
```
#Decomposing the time series

#Importing seasonal_decompose from statsmodels.tsa.seasonal

from statsmodels.tsa.seasonal import seasonal_decompose #Decompose Timeseries
```

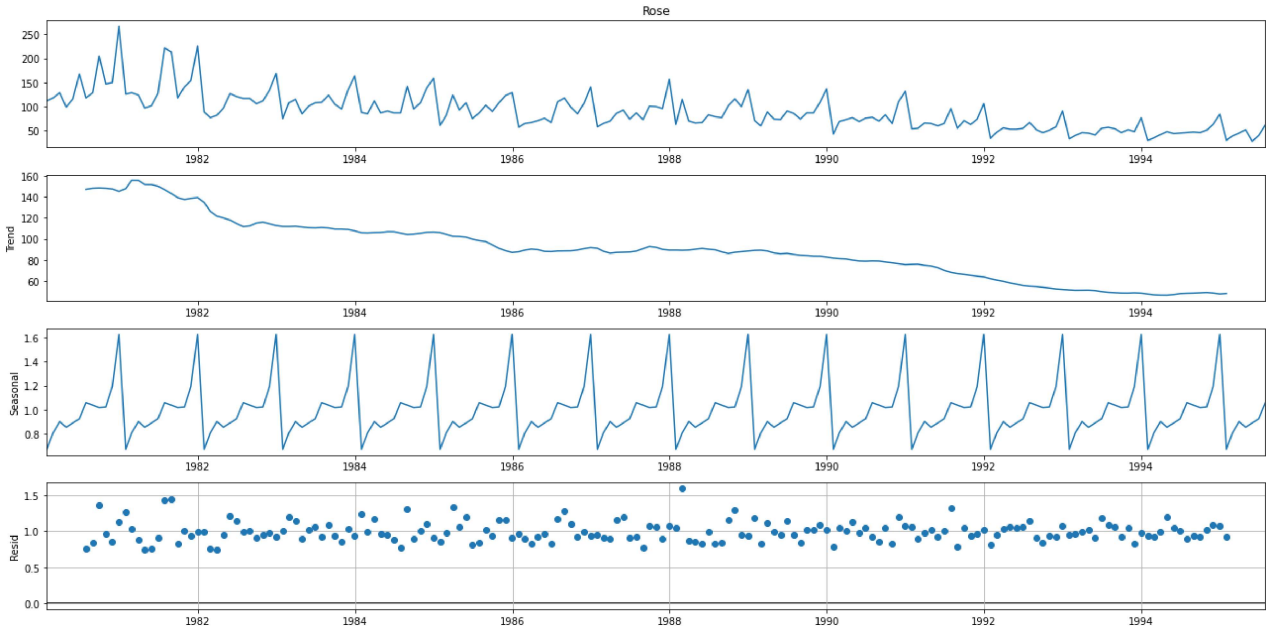
In [20]:
```
# Let's first do the Additive decomposition:

add_decomposition = seasonal_decompose(rose_data['Rose'], model = 'additive')
```

```
#plotting
add_decomposition.plot();
plt.grid()
plt.show()
```



From the above plot we can clearly say that residual showing some patterns in the data, but trend and seasonality has no growth over the time.

In [21]:
```
#Let's now look at the Multiplicative decomposition

mul_decomposition = seasonal_decompose(rose_data['Rose'], model = 'multiplicative')

mul_decomposition.plot();

plt.grid()
plt.show()
```



In the multiplicative model, residual still showing some patterns, so we will go with additive model.

3. Split the data into training and test. The test data should start in 1991.

In [22]:
```
#Let's split the data as per given condition

train = rose_data[rose_data.index.year < 1991]
test = rose_data[rose_data.index.year >= 1991]
```

In [23]:
```
#checking tail of the train set and head of the test set

train.tail()
```

Out[23]:

Time_stamp	Rose
1990-08-31	70.0
1990-09-30	83.0
1990-10-31	65.0
1990-11-30	110.0
1990-12-31	132.0

In [24]:
```
test.head()
```

Out[24]:

Time_stamp	Rose
1991-01-31	54.0
1991-02-28	55.0
1991-03-31	66.0
1991-04-30	65.0
1991-05-31	60.0

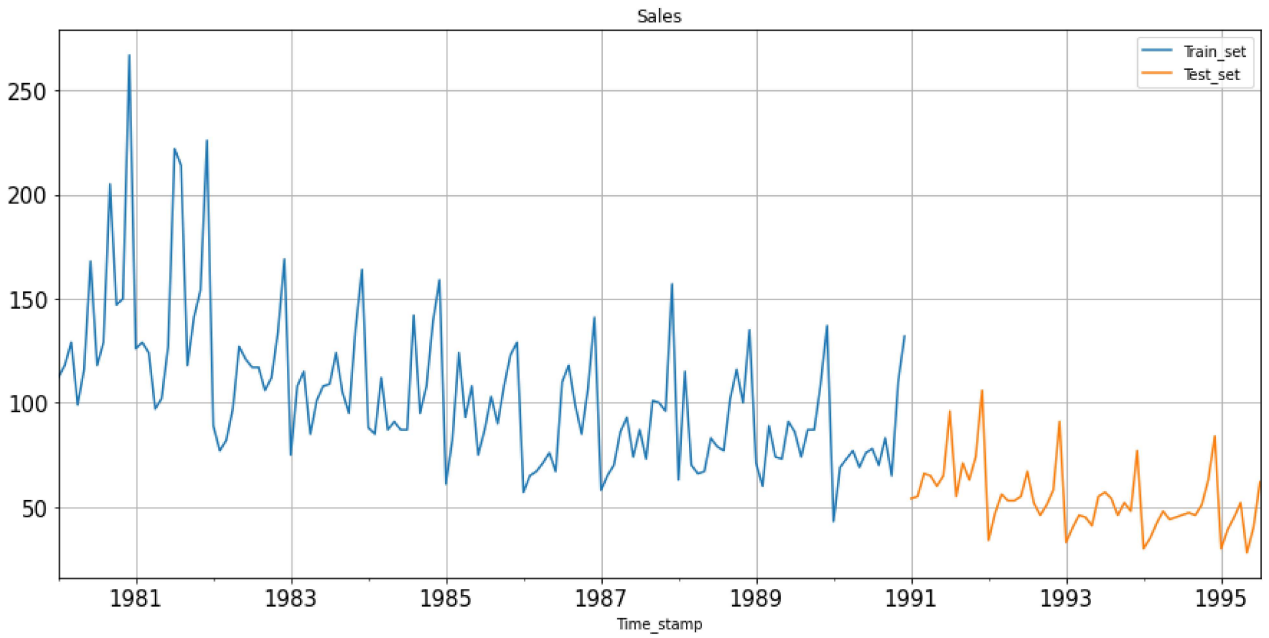
In [25]:
```
test.tail()
```

Out[25]:

Time_stamp	Rose
1995-03-31	45.0
1995-04-30	52.0
1995-05-31	28.0
1995-06-30	40.0
1995-07-31	62.0

We can see that the train set has the records up to 1990 and from 1991 test set holds the records up to 1995.

In [26]:
```
#now let' look at the plot for train and test set

train['Rose'].plot(figsize = (15,7), fontsize = 15)
test['Rose'].plot(figsize = (15,7), fontsize = 15)
plt.grid()
plt.legend(['Train_set','Test_set'])
plt.title('Sales')
plt.show()
```



4. Build various exponential smoothing models on the training data and evaluate the model using RMSE on the test data. Other models such as regression,naïve forecast models, simple average models etc. should also be built on the training data and check the performance on the test data using RMSE.

Building Linear Regression Model:

In [27]:
```
#modifying the train set before fitting into the model

train_time = [i+1 for i in range(len(train))]
test_time = [i+133 for i in range(len(test))]
print('Time instance for train set', '\n',train_time)
print('Time instance for test set', '\n',test_time)
```

```
Time instance for train set
 [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46,
47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90,
91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 11
0, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127,
128, 129, 130, 131, 132]
```

```
Time instance for test set
 [133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 1
50, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167,
168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 18
5, 186, 187]
```

In [28]:
```
#copying the train and test set for further analysis

LR_train = train.copy()
LR_test = test.copy()
```

In [29]:
```
#now creating an new columns name time_instance and test_instance and putting the value
LR_train['time_instance'] = train_time
LR_test['time_instance'] = test_time
```

In [30]:
```
#Let's check now

LR_train.head()
```

Out[30]:

Time_stamp	Rose	time_instance
1980-01-31	112.0	1
1980-02-29	118.0	2
1980-03-31	129.0	3
1980-04-30	99.0	4
1980-05-31	116.0	5

In [31]:
```
LR_test.head()
```

Out[31]:

Time_stamp	Rose	time_instance
1991-01-31	54.0	133
1991-02-28	55.0	134
1991-03-31	66.0	135
1991-04-30	65.0	136
1991-05-31	60.0	137

In [32]:
```
print(LR_train.shape)
print(LR_test.shape)
```

```
(132, 2)
(55, 2)
```

In [33]:

```
#now, let's import the LinearRegression package and fitting the train set

from sklearn.linear_model import LinearRegression

LR = LinearRegression()

LR.fit(LR_train[['time_instance']],LR_train['Rose'].values)
```

Out[33]:
```
LinearRegression()
```

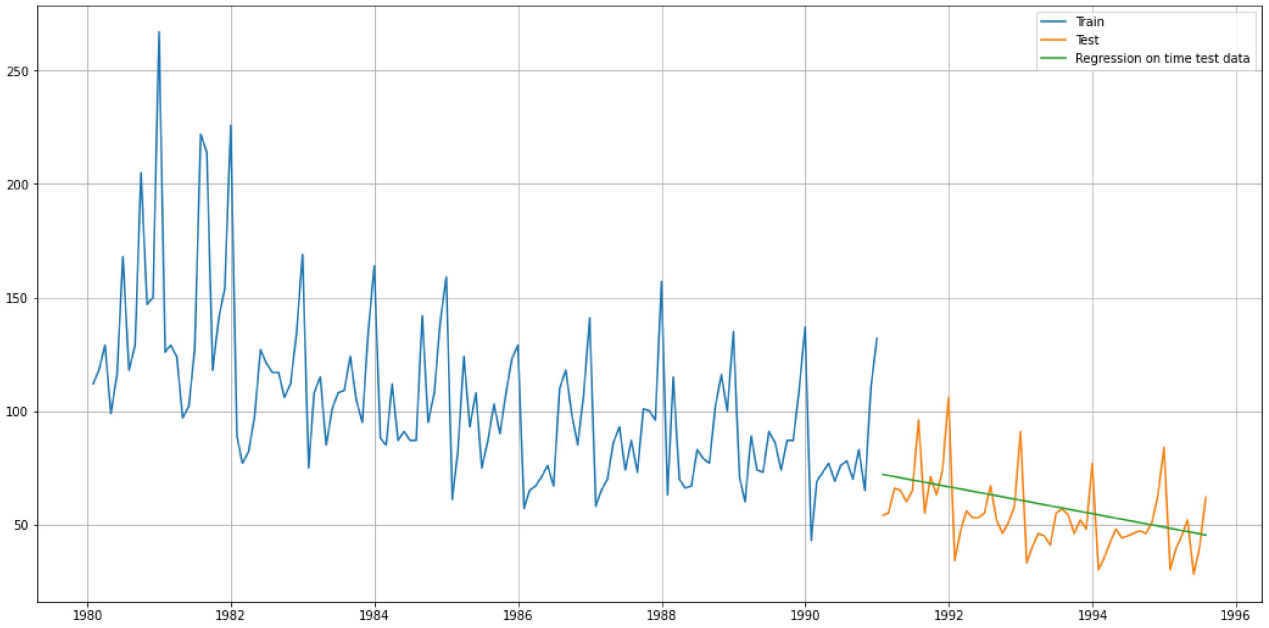
In [34]:
```
#Now, checking the predictions on test set

LR_test_predict = LR.predict(LR_test[['time_instance']])

#Now, adding one more column for 'Regression_on_time'  in test

LR_test['Regression_on_time'] = LR_test_predict
```

In [35]:
```
LR_test.head()
```

Out[35]:

Time_stamp	Rose	time_instance	Regression_on_time
1991-01-31	54.0	133	72.063266
1991-02-28	55.0	134	71.568888
1991-03-31	66.0	135	71.074511
1991-04-30	65.0	136	70.580133
1991-05-31	60.0	137	70.085755

In [36]:
```
#Plotting

plt.plot(train['Rose'],label = 'Train')
plt.plot(test['Rose'], label = 'Test')
plt.plot(LR_test['Regression_on_time'], label = 'Regression on time test data')

plt.legend(loc = 'best')
plt.grid()
```



In [37]:
```
#now, let's calculate the RMSE value on test data

from sklearn import metrics

#RMSE on test data
RMSE_LR_test = metrics.mean_squared_error(test['Rose'],LR_test_predict,squared=False)
print("RMSE for regression forecast on test data is%3.3f" %(RMSE_LR_test))


resultsDf = pd.DataFrame({'Test RMSE': [RMSE_LR_test]},index=['RegressionOnTime'])
resultsDf
```

```
RMSE for regression forecast on test data is15.255
```

Out[37]:

	Test RMSE
RegressionOnTime	15.255435

Naive model:

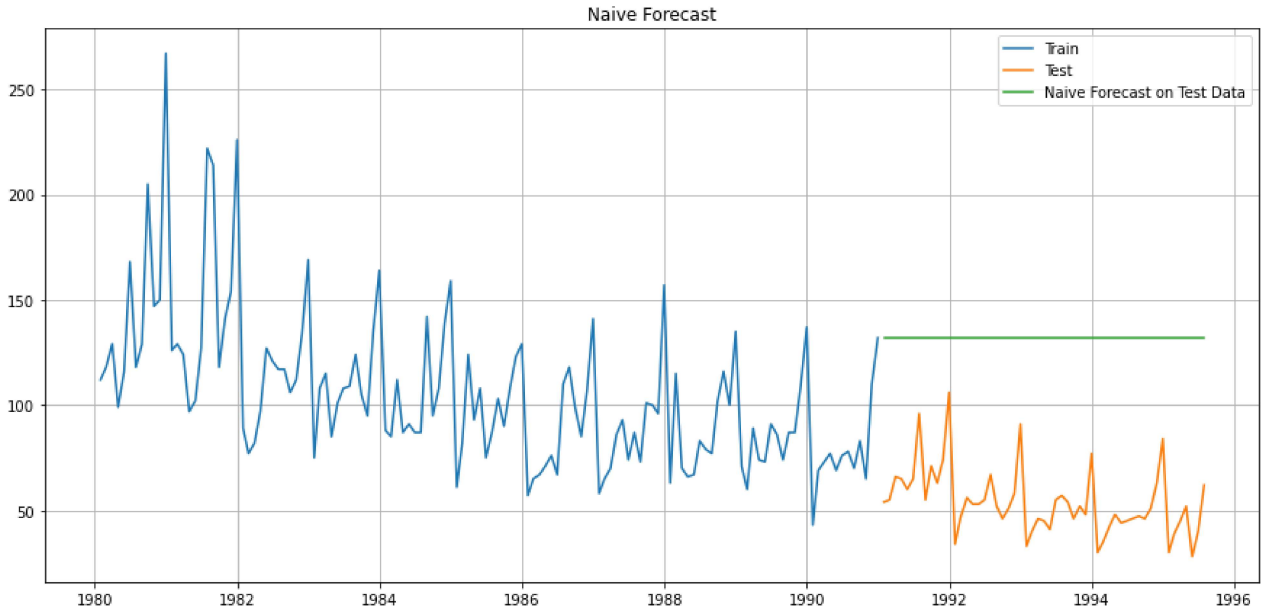
In [38]:
```
NM_train = train.copy()
NM_test = test.copy()
```

In [39]:
```
#now we will create the new column in the NM_test set and checking the head
NM_test['naive'] = np.asarray(train['Rose'])[len(np.asarray(train['Rose']))-1]
NM_test['naive'].head()
```

Out[39]:
```
Time_stamp
1991-01-31    132.0
1991-02-28    132.0
1991-03-31    132.0
1991-04-30    132.0
1991-05-31    132.0
Name: naive, dtype: float64
```

In [40]:
```
#Plotting
```

```
plt.figure(figsize=(15,7))

plt.plot(NM_train['Rose'], label='Train')
plt.plot(test['Rose'], label='Test')
plt.plot(NM_test['naive'], label='Naive Forecast on Test Data')
plt.legend(loc='best')
plt.title("Naive Forecast")
plt.grid()
plt.show()
```



In [41]:
```
#Let's check RMSE value on test data

#RMSE on test set
RMSE_NM_test = metrics.mean_squared_error(test['Rose'],NM_test['naive'],squared=False)
print("RMSE for naive model forecast on test data is %3.3f" %(RMSE_NM_test))
```

```
RMSE for naive model forecast on test data is 79.672
```

Simple Average Model:

In [42]:
```
#copying for further evaluation
SA_train = train.copy()
SA_test = test.copy()
```

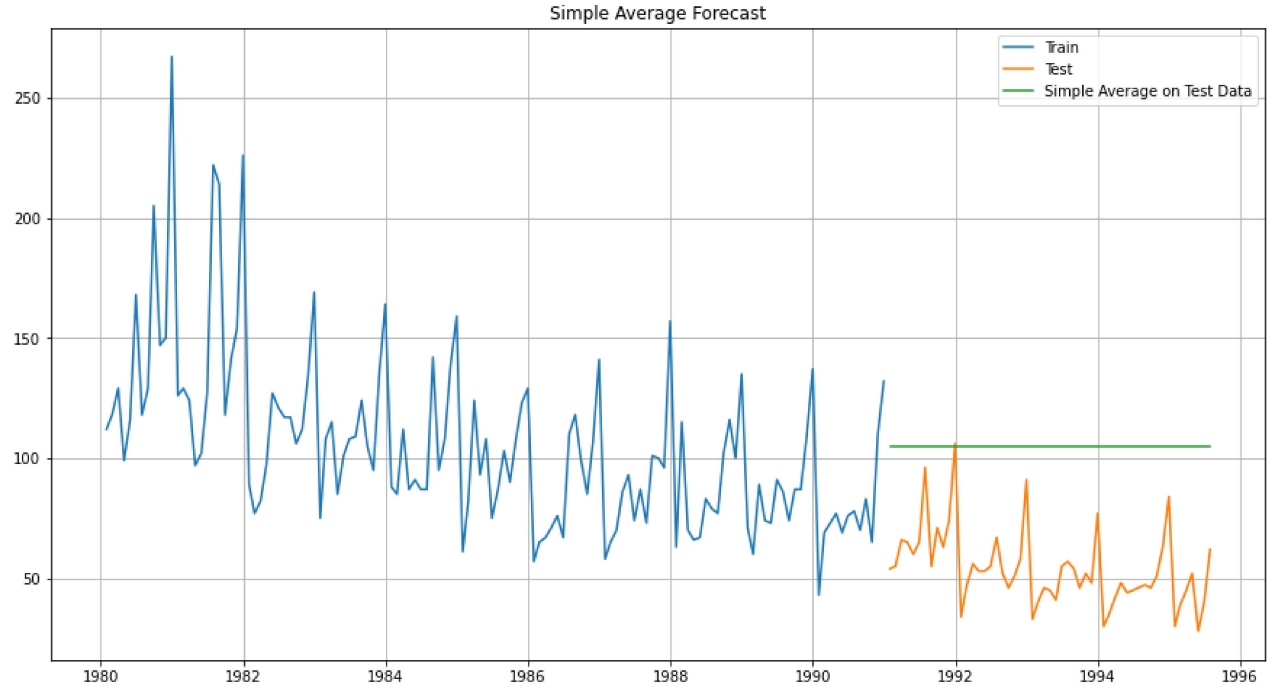
In [43]:
```
#In Simple Average model we will take the average of the training values and then will

SA_test['mean_forecast'] = train['Rose'].mean()

SA_test.head()
```

Out[43]:

Time_stamp	Rose	mean_forecast
1991-01-31	54.0	104.939394

Time_stamp	Rose	mean_forecast
1991-02-28	55.0	104.939394
1991-03-31	66.0	104.939394
1991-04-30	65.0	104.939394
1991-05-31	60.0	104.939394

In [44]:
```
#Plotting

plt.figure(figsize=(15,8))
plt.plot(SA_train['Rose'], label='Train')
plt.plot(SA_test['Rose'], label='Test')
plt.plot(SA_test['mean_forecast'], label='Simple Average on Test Data')
plt.legend(loc='best')
plt.title("Simple Average Forecast")
plt.grid();
```



In [45]:
```
#Calculating RMSE value on test data

RMSE_SA_test = metrics.mean_squared_error(test['Rose'],SA_test['mean_forecast'],squared
```

In [46]:
```
print("RMSE for Simple average forecast on test data is %3.3f" %(RMSE_SA_test))
```

```
RMSE for Simple average forecast on test data is 53.413
```

Till now, regression forecast is best with having the lowest rmse value of 15.25

Building Moving Average Model:

In [47]:

```
#For this model, we need to calculate moving averages for different intervals.

#The best interval can be determined by the maximum accuracy with the minimum error.

mov_avg = rose_data.copy()
```

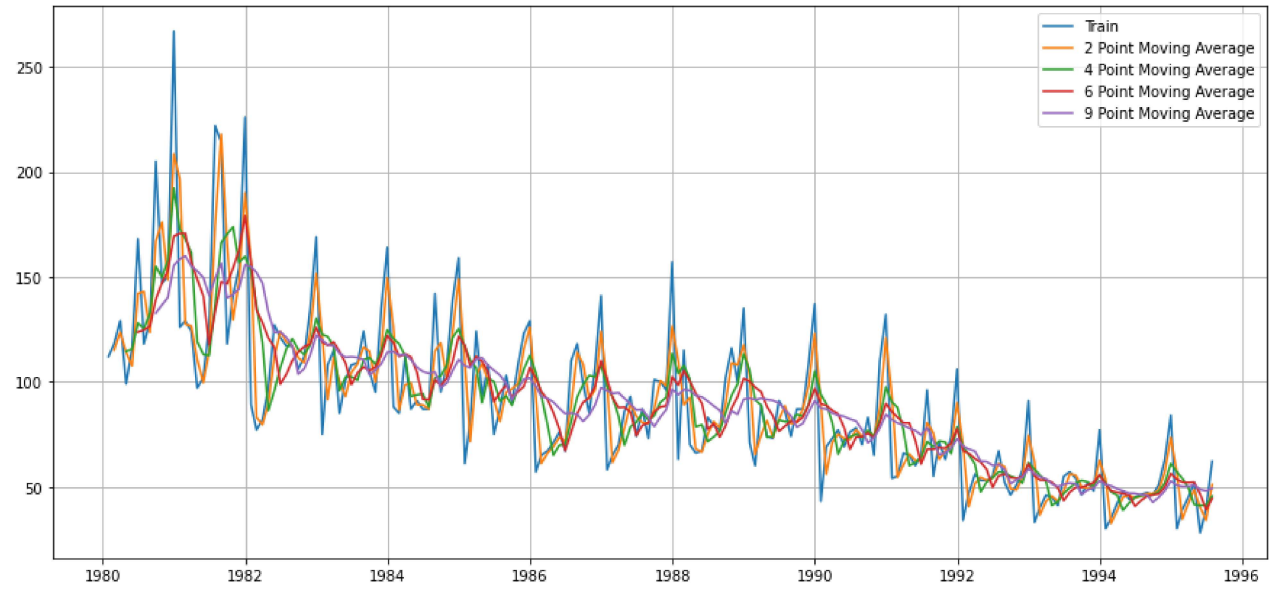
In [48]:
```
#Now, for this model, we'll calculate rolling means for different intervals.

#let's calculate this:

mov_avg['Trailing_2'] = mov_avg['Rose'].rolling(2).mean()
mov_avg['Trailing_4'] = mov_avg['Rose'].rolling(4).mean()
mov_avg['Trailing_6'] = mov_avg['Rose'].rolling(6).mean()
mov_avg['Trailing_9'] = mov_avg['Rose'].rolling(9).mean()
```

In [49]:
```
mov_avg.head()
```

Out[49]:

Time_stamp	Rose	Trailing_2	Trailing_4	Trailing_6	Trailing_9
1980-01-31	112.0	NaN	NaN	NaN	NaN
1980-02-29	118.0	115.0	NaN	NaN	NaN
1980-03-31	129.0	123.5	NaN	NaN	NaN
1980-04-30	99.0	114.0	114.5	NaN	NaN
1980-05-31	116.0	107.5	115.5	NaN	NaN

In [50]:
```
#Plotting

plt.figure(figsize=(15,7))
plt.plot(mov_avg['Rose'], label='Train')
plt.plot(mov_avg['Trailing_2'], label='2 Point Moving Average')
plt.plot(mov_avg['Trailing_4'], label='4 Point Moving Average')
plt.plot(mov_avg['Trailing_6'],label = '6 Point Moving Average')
plt.plot(mov_avg['Trailing_9'],label = '9 Point Moving Average')
plt.legend(loc = 'best')
plt.grid()
plt.show()
```



In [51]:
```
# Now we will split the data and create the train and test set

trailing_MA_train= mov_avg[0:int(len(mov_avg)*0.709677)]
trailing_MA_test = mov_avg[int(len(mov_avg)*0.709677):]
```

In [52]:
```
#Now, let's check the count for trailing 2, trailing 4, trailing 6 and trailing 9 movin

trailing_MA_train.count()
```

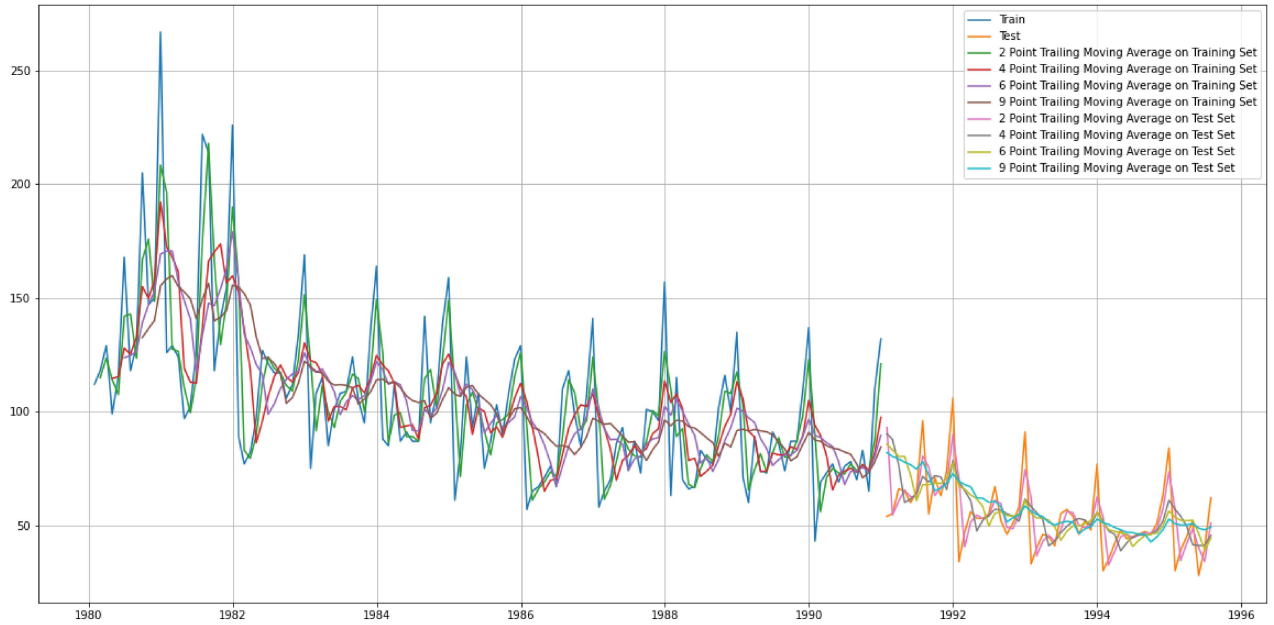
Out[52]:
```
Rose          132
Trailing_2    131
Trailing_4    129
Trailing_6    127
Trailing_9    124
dtype: int64
```

In [53]:
```
#Now, let's check the count for trailing 2, trailing 4, trailing 6 and trailing 9 movin

trailing_MA_test.count()
```

Out[53]:
```
Rose          55
Trailing_2    55
Trailing_4    55
Trailing_6    55
Trailing_9    55
dtype: int64
```

In [54]:
```
#Plotting

plt.figure(figsize=(20,10))
plt.plot(trailing_MA_train['Rose'], label='Train')
plt.plot(trailing_MA_test['Rose'], label='Test')

plt.plot(trailing_MA_train['Trailing_2'],label='2 Point Trailing Moving Average on Trai
plt.plot(trailing_MA_train['Trailing_4'],label='4 Point Trailing Moving Average on Trai
plt.plot(trailing_MA_train['Trailing_6'],label= '6 Point Trailing Moving Average on Tra
plt.plot(trailing_MA_train['Trailing_9'],label= '9 Point Trailing Moving Average on Tra
```

```
plt.plot(trailing_MA_test['Trailing_2'],label='2 Point Trailing Moving Average on Test
plt.plot(trailing_MA_test['Trailing_4'],label='4 Point Trailing Moving Average on Test
plt.plot(trailing_MA_test['Trailing_6'],label= '6 Point Trailing Moving Average on Test
plt.plot(trailing_MA_test['Trailing_9'],label= '9 Point Trailing Moving Average on Test
plt.legend(loc = 'best')
plt.grid()
plt.show()
```



In [55]:

```
#Evaluation on test data for RMSE values of trailing 2,4,6 and 9

#RMSE for Test Data :2 point Trailing MA

RMSE_MA_test_2 = metrics.mean_squared_error(test['Rose'],trailing_MA_test['Trailing_2']
print("For 2 point Moving Average Model forecast on the Training Data,  RMSE value is %

#RMSE for Test Data :4 point Trailing MA

RMSE_MA_test_4 = metrics.mean_squared_error(test['Rose'],trailing_MA_test['Trailing_4']
print("For 4 point Moving Average Model forecast on the Training Data,  RMSE value is %

#RMSE for Test Data :6 point Trailing MA

RMSE_MA_test_6 = metrics.mean_squared_error(test['Rose'],trailing_MA_test['Trailing_6']
print("For 6 point Moving Average Model forecast on the Training Data,  RMSE value is %

#RMSE for Test Data :9 point Trailing MA

RMSE_MA_test_9 = metrics.mean_squared_error(test['Rose'],trailing_MA_test['Trailing_9']
print("For 9 point Moving Average Model forecast on the Training Data,  RMSE value is %
```

```
For 2 point Moving Average Model forecast on the Training Data,  RMSE value is 11.530
For 4 point Moving Average Model forecast on the Training Data,  RMSE value is 14.444
For 6 point Moving Average Model forecast on the Training Data,  RMSE value is 14.555
For 9 point Moving Average Model forecast on the Training Data,  RMSE value is 14.721
```

From the above calculations, we can say that till now the best model seems to be 2 point moving average forecast model, with the lowest rmse value of 11.530

Building Triple Exponential Smoothing model or Holt-Winter's model:

Since our data shows trend and seasonality patterns, it will good to use Triple Exponential smoothing model for this data.

In [56]:
```
#importing ExponentialSmoothing, SimpleExpSmoothing and Holt

from statsmodels.tsa.api import ExponentialSmoothing, SimpleExpSmoothing, Holt

#copying the train and test set

TES_train = train.copy()
TES_test = test.copy()

#creating class

TES_model = ExponentialSmoothing(TES_train['Rose'],trend='additive',seasonal='multiplic

# Fitting the Simple Exponential Smoothing model and asking python to choose the optima
TES_model_autofit = TES_model.fit(optimized=True)
```

In [57]:
```
#checking the autofit parameters

TES_model_autofit.params
```

Out[57]:
```
{'smoothing_level': 0.06569374607191865,
 'smoothing_trend': 0.05192938504457338,
 'smoothing_seasonal': 3.879136202038614e-06,
 'damping_trend': nan,
 'initial_level': 54.10985491750761,
 'initial_trend': -0.33471965714896845,
 'initial_seasons': array([2.08282313, 2.36326666, 2.58210206, 2.25702695, 2.53757493,
        2.76639991, 3.04101803, 3.23434567, 3.06747277, 3.00164124,
        3.49893806, 4.82552476]),
 'use_boxcox': False,
 'lamda': None,
 'remove_bias': False}
```

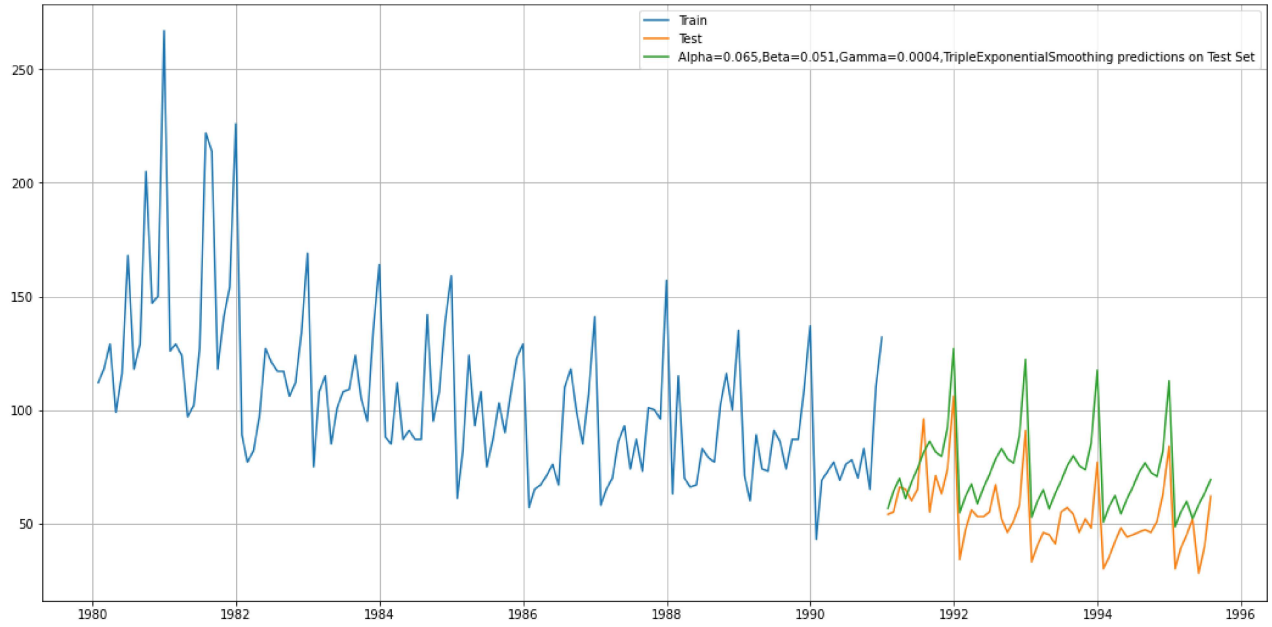
In [58]:
```
## Prediction on the test data
TES_test['auto_predict'] = TES_model_autofit.forecast(steps=len(test))
TES_test.head()
```

Out[58]:

Time_stamp	Rose	auto_predict
1991-01-31	54.0	56.689174
1991-02-28	55.0	64.129166
1991-03-31	66.0	69.856436
1991-04-30	65.0	60.877474
1991-05-31	60.0	68.237072

In [59]:
```
## Plotting on Test data using autofit
plt.figure(figsize=(18,9))
plt.plot(TES_train['Rose'], label='Train')
```

```
plt.plot(TES_test['Rose'], label='Test')
plt.plot(TES_test['auto_predict'], label='Alpha=0.065,Beta=0.051,Gamma=0.0004,TripleExp
plt.legend(loc='best')
plt.grid();
```



In [60]:

```
## Test Data
RMSE_TES_test = metrics.mean_squared_error(TES_test['Rose'],TES_test['auto_predict'],sq
print("For Alpha=0.065,Beta=0.051,Gamma=0.0004, Triple Exponential Smoothing Model fore
```

```
For Alpha=0.065,Beta=0.051,Gamma=0.0004, Triple Exponential Smoothing Model forecast on
the Test Data,  RMSE is 20.960
```

In [61]:

```
#Let's try with other alpha, beta and gamma values

## First we will define an empty dataframe to store our values from the loop
resultsDf_new = pd.DataFrame({'Alpha Values':[],'Beta Values':[],'Gamma Values':[],'Tra
resultsDf_new

for i in np.arange(0.3,1.1,0.1):
    for j in np.arange(0.3,1.1,0.1):
        for k in np.arange(0.3,1.1,0.1):
            TES_model_alpha_i_j_k = TES_model.fit(smoothing_level=i,smoothing_trend=j,s
            TES_train['predict',i,j,k] = TES_model_alpha_i_j_k.fittedvalues
            TES_test['predict',i,j,k] = TES_model_alpha_i_j_k.forecast(steps=55)

            RMSE_TES_train_new = metrics.mean_squared_error(TES_train['Rose'],TES_train

            RMSE_TES_test_new = metrics.mean_squared_error(TES_test['Rose'],TES_test['p

            resultsDf_new = resultsDf_new.append({'Alpha Values':i,'Beta Values':j,'Gam
                                                  'Train RMSE':RMSE_TES_train_new,'Test
                                                 , ignore_index=True)
```

In [62]:

```
#Let's check out the top head values with least RMSE values

resultsDf_new.sort_values(by=['Test RMSE']).head()
```

Out[62]:

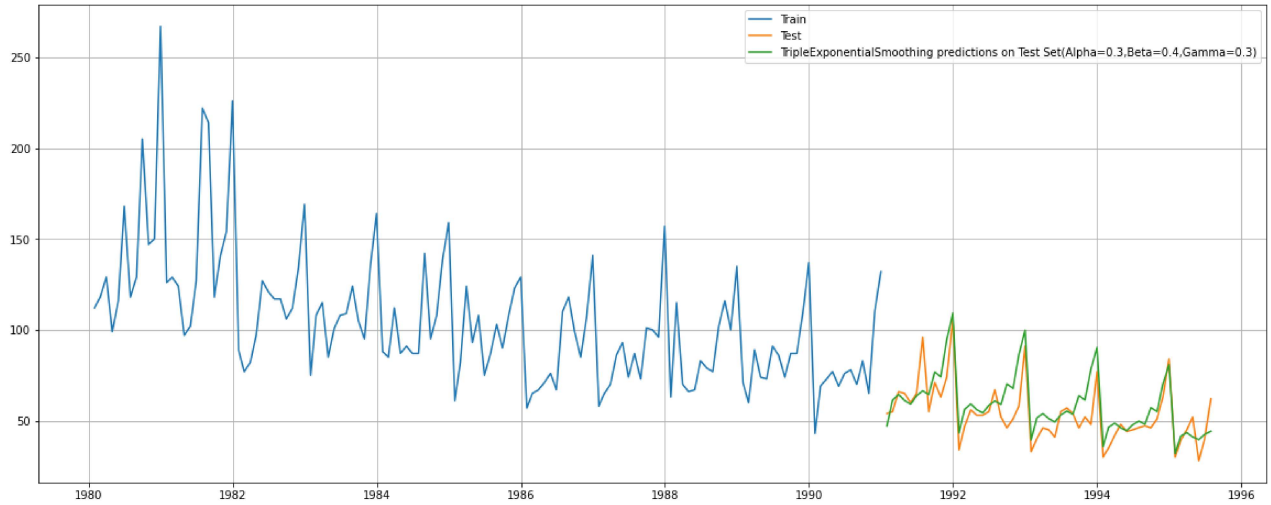
	Alpha Values	Beta Values	Gamma Values	Train RMSE	Test RMSE
8	0.3	0.4	0.3	28.111886	10.935749
1	0.3	0.3	0.4	27.399095	11.243009
69	0.4	0.3	0.8	32.601491	12.624534
16	0.3	0.5	0.3	29.087520	14.461672
131	0.5	0.3	0.6	32.144773	16.762882

In [63]:

```
#we can see that at alpha value 0.3, beta value 0.4 and gamma value 0.3, we are getting

RMSE_TES_test_new_1 = 10.935749
```

In [64]:

```
#now let's plot the model with these values

plt.figure(figsize=(20,8))
plt.plot(TES_train['Rose'], label='Train')
plt.plot(TES_test['Rose'], label='Test')
plt.plot(TES_test['predict', 0.3, 0.4, 0.3], label= 'TripleExponentialSmoothing predict
plt.legend(loc='best')
plt.grid()
plt.show()
```



Green line on the plot shows the TES predictions on test set with best alpha, beta and gamma values.

5. Check for the stationarity of the data on which the model is being built on using appropriate statistical tests and also mention the hypothesis for the statistical test. If the data is found to be non-stationary, take appropriate steps to make it stationary. Check the new data for stationarity and comment. Note: Stationarity should be checked at alpha = 0.05.

we will check for the stationarity of the data by sing Dickey Fuller test

Dickey Fuller Test:

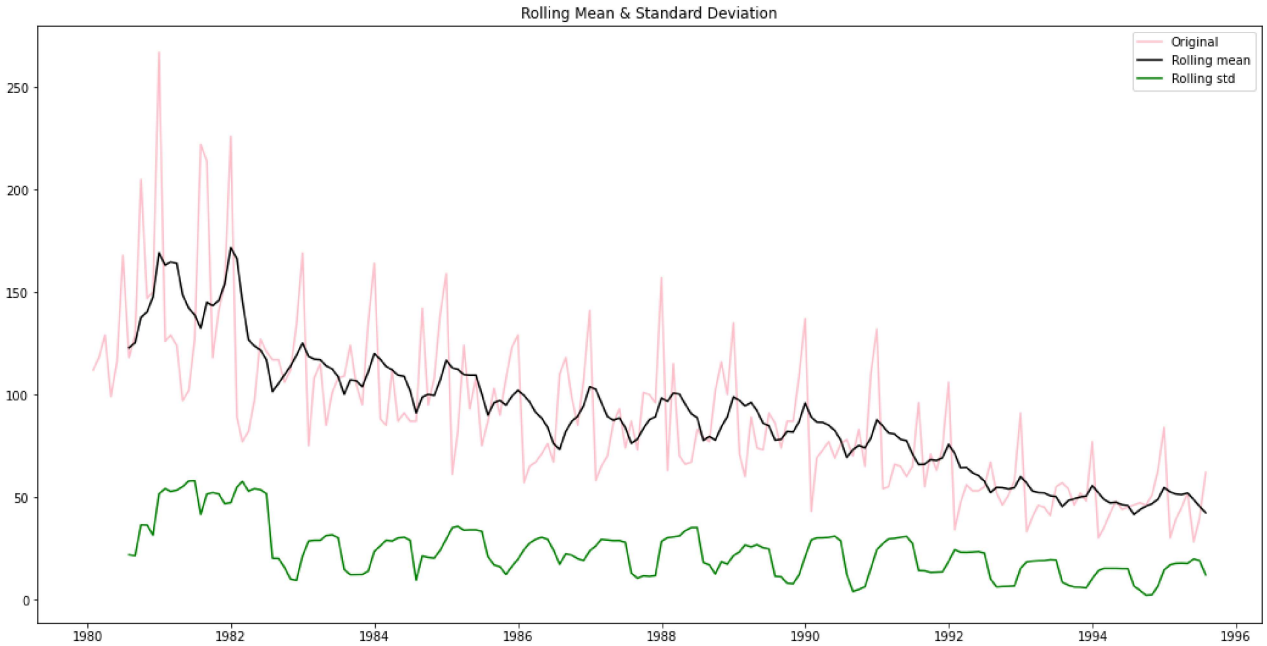
Taking Null Hypothesis H_0- Series is not Stationary

Taking Alternative Hypothesis H_1- Series is Stationary

In [65]:
```
#importing adfuller

from statsmodels.tsa.stattools import adfuller
```

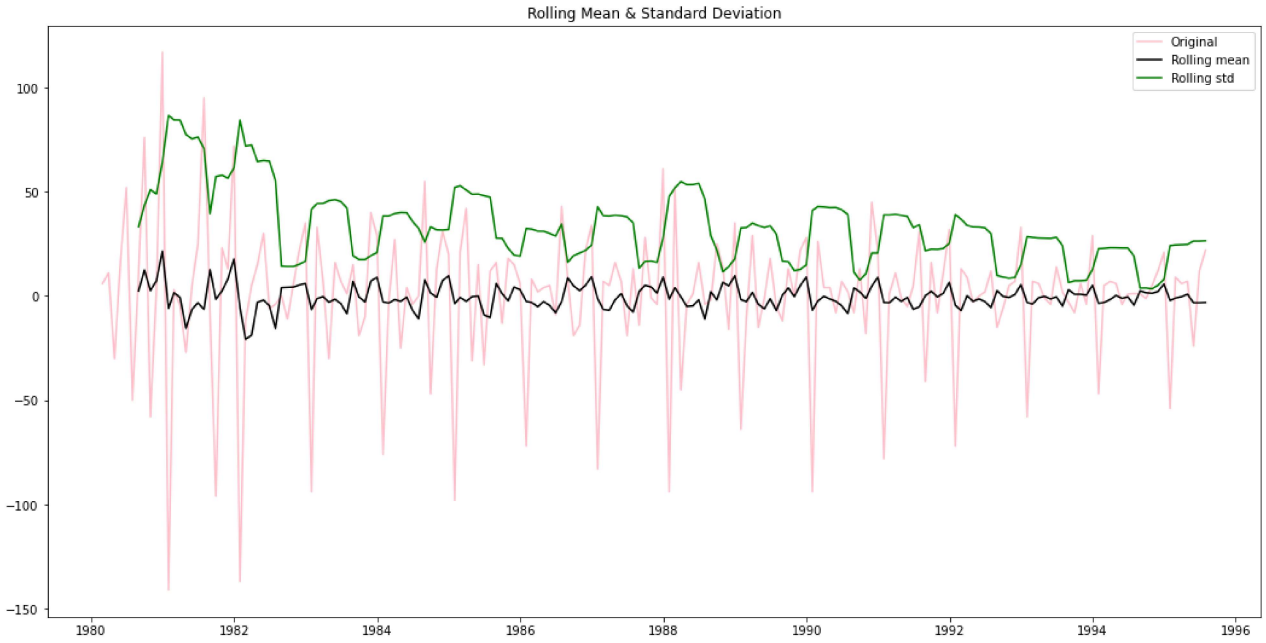
In [66]:
```
#creating a function to test the stationarity of the time series


def Stationarity_test(timeseries):

    #rolling mean
    rolmean = timeseries.rolling(window=7).mean()
    #rolling standard deviation
    rolstd = timeseries.rolling(window=7).std()

    #Plotting

    original = plt.plot(timeseries, color='pink',label='Original')
    mean = plt.plot(rolmean, color='black', label='Rolling mean')
    std = plt.plot(rolstd, color='green', label = 'Rolling std')
    plt.legend(loc='best')
    plt.title('Rolling Mean & Standard Deviation')
    plt.show(block=False)
```

In [67]:
```
#calling the function

Stationarity_test(rose_data['Rose'])
```



Time series isn't stationary at 5% significant level, let's now check with difference of order 1.

In [68]:
```
Stationarity_test(rose_data['Rose'].diff().dropna())
```



Series is now stationary.

6. Build an automated version of the ARIMA/SARIMA model in which the parameters are selected using the lowest Akaike Information Criteria (AIC) on the training data and evaluate this model on the test data using RMSE.

In [69]:
```
#we will take the combination of different parameters of p and q in the range of 0 and

#we have kept the value of d as 1 as we need to take a difference of the series to make

#importing itertools

import itertools
p = q = range(0, 3)
d= range(1,2)
pdq = list(itertools.product(p, d, q))
print('Combination of parameters for the model:')
for i in range(1,len(pdq)):
    print('Model: {}'.format(pdq[i]))
```

```
Combination of parameters for the model:
Model: (0, 1, 1)
Model: (0, 1, 2)
Model: (1, 1, 0)
Model: (1, 1, 1)
Model: (1, 1, 2)
Model: (2, 1, 0)
Model: (2, 1, 1)
Model: (2, 1, 2)
```

In [70]:
```
# Creating an empty Dataframe with column names only
ARIMA_AIC = pd.DataFrame(columns=['param', 'AIC'])
ARIMA_AIC
```

Out[70]:

	param	AIC

In [71]:
```
#importing ARIMA

from statsmodels.tsa.arima_model import ARIMA

for param in pdq:
    ARIMA_model = ARIMA(train['Rose'].values,order=param).fit()
    print('ARIMA{} - AIC:{}'.format(param,ARIMA_model.aic))
    ARIMA_AIC = ARIMA_AIC.append({'param':param, 'AIC': ARIMA_model.aic}, ignore_index=
```

```
ARIMA(0, 1, 0) - AIC:1335.1526583086775
ARIMA(0, 1, 1) - AIC:1280.726183046448
ARIMA(0, 1, 2) - AIC:1276.8353734911866
ARIMA(1, 1, 0) - AIC:1319.3483105802602
ARIMA(1, 1, 1) - AIC:1277.775753553521
ARIMA(1, 1, 2) - AIC:1277.3592281129256
ARIMA(2, 1, 0) - AIC:1300.6092611744193
ARIMA(2, 1, 1) - AIC:1279.0456894093354
ARIMA(2, 1, 2) - AIC:1279.2986939365205
```

The lowest AIC value comes out to 1276.83 at ARIMA(0,1,2)

In [72]:
```
#Now, let's create auto ARIMA by using these parameters

auto_ARIMA = ARIMA(train['Rose'], order=(0,1,2))
results_auto_ARIMA = auto_ARIMA.fit()
print(results_auto_ARIMA.summary())
```

```
                             ARIMA Model Results
==============================================================================
Dep. Variable:                 D.Rose   No. Observations:                  131
Model:                 ARIMA(0, 1, 2)   Log Likelihood                -634.418
Method:                       css-mle   S.D. of innovations             30.167
Date:                Mon, 11 Oct 2021   AIC                           1276.835
Time:                        23:16:13   BIC                           1288.336
Sample:                    02-29-1980   HQIC                          1281.509
                         - 12-31-1990
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          -0.4886      0.085     -5.742      0.000      -0.655      -0.322
ma.L1.D.Rose   -0.7601      0.101     -7.499      0.000      -0.959      -0.561
ma.L2.D.Rose   -0.2398      0.095     -2.518      0.012      -0.427      -0.053
                                    Roots
=============================================================================
                  Real          Imaginary           Modulus         Frequency
-----------------------------------------------------------------------------
MA.1            1.0001           +0.0000j            1.0001            0.0000
MA.2           -4.1695           +0.0000j            4.1695            0.5000
-----------------------------------------------------------------------------
```

In [73]:
```
#prediction on test set

predicted_auto_ARIMA = results_auto_ARIMA.forecast(steps=len(test))
```

In [74]:

```
#importing mean_squared_error

from sklearn.metrics import  mean_squared_error
```

In [75]:
```
#Calculating RMSE on test data

RMSE_1 = mean_squared_error(test['Rose'],predicted_auto_ARIMA[0],squared=False)
```

In [76]:
```
print(RMSE_1)
```

```
15.60419552386519
```

Automated SARIMA

In [77]:
```
import itertools
p = q = range(0, 3)
d= range(1,2)
D = range(0,1)
pdq = list(itertools.product(p, d, q))
model_pdq = [(x[0], x[1], x[2], 12) for x in list(itertools.product(p, D, q))]
print('Combinations of parameter for the SARIMA model:')
for i in range(1,len(pdq)):
    print('Model: {}{}'.format(pdq[i], model_pdq[i]))
```

```
Combinations of parameter for the SARIMA model:
Model: (0, 1, 1)(0, 0, 1, 12)
Model: (0, 1, 2)(0, 0, 2, 12)
Model: (1, 1, 0)(1, 0, 0, 12)
Model: (1, 1, 1)(1, 0, 1, 12)
Model: (1, 1, 2)(1, 0, 2, 12)
Model: (2, 1, 0)(2, 0, 0, 12)
Model: (2, 1, 1)(2, 0, 1, 12)
Model: (2, 1, 2)(2, 0, 2, 12)
```

In [78]:
```
SARIMA_AIC = pd.DataFrame(columns=['param','seasonal', 'AIC'])
SARIMA_AIC
```

Out[78]:

	param	seasonal	AIC

In [79]:
```
import statsmodels.api as sm

for param in pdq:
    for param_seasonal in model_pdq:
        SARIMA_model = sm.tsa.statespace.SARIMAX(train['Rose'].values,
                                                 order=param,
                                                 seasonal_order=param_seasonal,
                                                 enforce_stationarity=False,
                                                 enforce_invertibility=False)

        results_SARIMA = SARIMA_model.fit(maxiter=1000)
        print('SARIMA{}x{} - AIC:{}'.format(param, param_seasonal, results_SARIMA.aic))
        SARIMA_AIC = SARIMA_AIC.append({'param':param,'seasonal':param_seasonal ,'AIC':
```

```
SARIMA(0, 1, 0)x(0, 0, 0, 12) - AIC:1323.9657875279158
```

```
SARIMA(0, 1, 0)x(0, 0, 1, 12) - AIC:1145.4230827207175
SARIMA(0, 1, 0)x(0, 0, 2, 12) - AIC:976.4375296380895
SARIMA(0, 1, 0)x(1, 0, 0, 12) - AIC:1139.921738995602
SARIMA(0, 1, 0)x(1, 0, 1, 12) - AIC:1116.0207869386002
SARIMA(0, 1, 0)x(1, 0, 2, 12) - AIC:969.6913635750373
SARIMA(0, 1, 0)x(2, 0, 0, 12) - AIC:960.8812220353041
SARIMA(0, 1, 0)x(2, 0, 1, 12) - AIC:962.8794540697546
SARIMA(0, 1, 0)x(2, 0, 2, 12) - AIC:955.5735408945867
SARIMA(0, 1, 1)x(0, 0, 0, 12) - AIC:1263.5369097383964
SARIMA(0, 1, 1)x(0, 0, 1, 12) - AIC:1098.5554825918339
SARIMA(0, 1, 1)x(0, 0, 2, 12) - AIC:923.6314049383807
SARIMA(0, 1, 1)x(1, 0, 0, 12) - AIC:1095.793632491795
SARIMA(0, 1, 1)x(1, 0, 1, 12) - AIC:1054.743433094548
SARIMA(0, 1, 1)x(1, 0, 2, 12) - AIC:918.8573483311407
SARIMA(0, 1, 1)x(2, 0, 0, 12) - AIC:914.5982866536075
SARIMA(0, 1, 1)x(2, 0, 1, 12) - AIC:915.3332430461666
SARIMA(0, 1, 1)x(2, 0, 2, 12) - AIC:901.1988257864924
SARIMA(0, 1, 2)x(0, 0, 0, 12) - AIC:1251.667543054104
SARIMA(0, 1, 2)x(0, 0, 1, 12) - AIC:1083.4866975264913
SARIMA(0, 1, 2)x(0, 0, 2, 12) - AIC:913.49384866177
SARIMA(0, 1, 2)x(1, 0, 0, 12) - AIC:1088.8332843413673
SARIMA(0, 1, 2)x(1, 0, 1, 12) - AIC:1045.540093355627
SARIMA(0, 1, 2)x(1, 0, 2, 12) - AIC:904.8310913510973
SARIMA(0, 1, 2)x(2, 0, 0, 12) - AIC:913.0105912257978
SARIMA(0, 1, 2)x(2, 0, 1, 12) - AIC:914.1707545034652
SARIMA(0, 1, 2)x(2, 0, 2, 12) - AIC:887.9375085679284
SARIMA(1, 1, 0)x(0, 0, 0, 12) - AIC:1308.161871082466
SARIMA(1, 1, 0)x(0, 0, 1, 12) - AIC:1135.2955447585719
SARIMA(1, 1, 0)x(0, 0, 2, 12) - AIC:963.9405391257686
SARIMA(1, 1, 0)x(1, 0, 0, 12) - AIC:1124.886078680458
SARIMA(1, 1, 0)x(1, 0, 1, 12) - AIC:1105.4080055024756
SARIMA(1, 1, 0)x(1, 0, 2, 12) - AIC:958.5001972948405
SARIMA(1, 1, 0)x(2, 0, 0, 12) - AIC:939.0984778664119
SARIMA(1, 1, 0)x(2, 0, 1, 12) - AIC:940.9087133661067
SARIMA(1, 1, 0)x(2, 0, 2, 12) - AIC:942.2973103071082
SARIMA(1, 1, 1)x(0, 0, 0, 12) - AIC:1262.1840064255505
SARIMA(1, 1, 1)x(0, 0, 1, 12) - AIC:1094.317270864085
SARIMA(1, 1, 1)x(0, 0, 2, 12) - AIC:923.0862224063868
SARIMA(1, 1, 1)x(1, 0, 0, 12) - AIC:1083.3937965032806
SARIMA(1, 1, 1)x(1, 0, 1, 12) - AIC:1054.7180547135129
SARIMA(1, 1, 1)x(1, 0, 2, 12) - AIC:916.3549428500592
SARIMA(1, 1, 1)x(2, 0, 0, 12) - AIC:905.924906084468
SARIMA(1, 1, 1)x(2, 0, 1, 12) - AIC:907.2972867470745
SARIMA(1, 1, 1)x(2, 0, 2, 12) - AIC:900.6725795936856
SARIMA(1, 1, 2)x(0, 0, 0, 12) - AIC:1251.9495040706236
SARIMA(1, 1, 2)x(0, 0, 1, 12) - AIC:1085.4861928100865
SARIMA(1, 1, 2)x(0, 0, 2, 12) - AIC:915.4938413564623
SARIMA(1, 1, 2)x(1, 0, 0, 12) - AIC:1090.7760922493494
SARIMA(1, 1, 2)x(1, 0, 1, 12) - AIC:1042.6183220083724
SARIMA(1, 1, 2)x(1, 0, 2, 12) - AIC:906.731849990849
SARIMA(1, 1, 2)x(2, 0, 0, 12) - AIC:906.1690197403764
SARIMA(1, 1, 2)x(2, 0, 1, 12) - AIC:907.4597827986624
SARIMA(1, 1, 2)x(2, 0, 2, 12) - AIC:889.9026503732757
SARIMA(2, 1, 0)x(0, 0, 0, 12) - AIC:1280.253756153577
SARIMA(2, 1, 0)x(0, 0, 1, 12) - AIC:1128.777370471139
SARIMA(2, 1, 0)x(0, 0, 2, 12) - AIC:958.0793208829958
SARIMA(2, 1, 0)x(1, 0, 0, 12) - AIC:1099.5086021575974
SARIMA(2, 1, 0)x(1, 0, 1, 12) - AIC:1076.786319864115
SARIMA(2, 1, 0)x(1, 0, 2, 12) - AIC:951.1988165559238
SARIMA(2, 1, 0)x(2, 0, 0, 12) - AIC:924.6004792645396
SARIMA(2, 1, 0)x(2, 0, 1, 12) - AIC:925.9757801384721
```

```
SARIMA(2, 1, 0)x(2, 0, 2, 12) - AIC:927.8380693280761
SARIMA(2, 1, 1)x(0, 0, 0, 12) - AIC:1263.2315231797984
SARIMA(2, 1, 1)x(0, 0, 1, 12) - AIC:1094.2093491949367
SARIMA(2, 1, 1)x(0, 0, 2, 12) - AIC:922.9408472110376
SARIMA(2, 1, 1)x(1, 0, 0, 12) - AIC:1071.4249601101342
SARIMA(2, 1, 1)x(1, 0, 1, 12) - AIC:1052.9244471204804
SARIMA(2, 1, 1)x(1, 0, 2, 12) - AIC:916.242491282262
SARIMA(2, 1, 1)x(2, 0, 0, 12) - AIC:896.5181608119215
SARIMA(2, 1, 1)x(2, 0, 1, 12) - AIC:897.639956536955
SARIMA(2, 1, 1)x(2, 0, 2, 12) - AIC:899.4835866288954
SARIMA(2, 1, 2)x(0, 0, 0, 12) - AIC:1253.9102116146685
SARIMA(2, 1, 2)x(0, 0, 1, 12) - AIC:1085.964355259821
SARIMA(2, 1, 2)x(0, 0, 2, 12) - AIC:916.3258311103143
SARIMA(2, 1, 2)x(1, 0, 0, 12) - AIC:1073.2912713704332
SARIMA(2, 1, 2)x(1, 0, 1, 12) - AIC:1044.1909354771824
SARIMA(2, 1, 2)x(1, 0, 2, 12) - AIC:907.666148879695
SARIMA(2, 1, 2)x(2, 0, 0, 12) - AIC:897.3464441348852
SARIMA(2, 1, 2)x(2, 0, 1, 12) - AIC:898.3781889114674
SARIMA(2, 1, 2)x(2, 0, 2, 12) - AIC:890.6687980804124
```

In [80]:

```
#Let's check top 5 lowest AIC values

SARIMA_AIC.sort_values(by=['AIC']).head()
```

Out[80]:

	param	seasonal	AIC
26	(0, 1, 2)	(2, 0, 2, 12)	887.937509
53	(1, 1, 2)	(2, 0, 2, 12)	889.902650
80	(2, 1, 2)	(2, 0, 2, 12)	890.668798
69	(2, 1, 1)	(2, 0, 0, 12)	896.518161
78	(2, 1, 2)	(2, 0, 0, 12)	897.346444

In [81]:

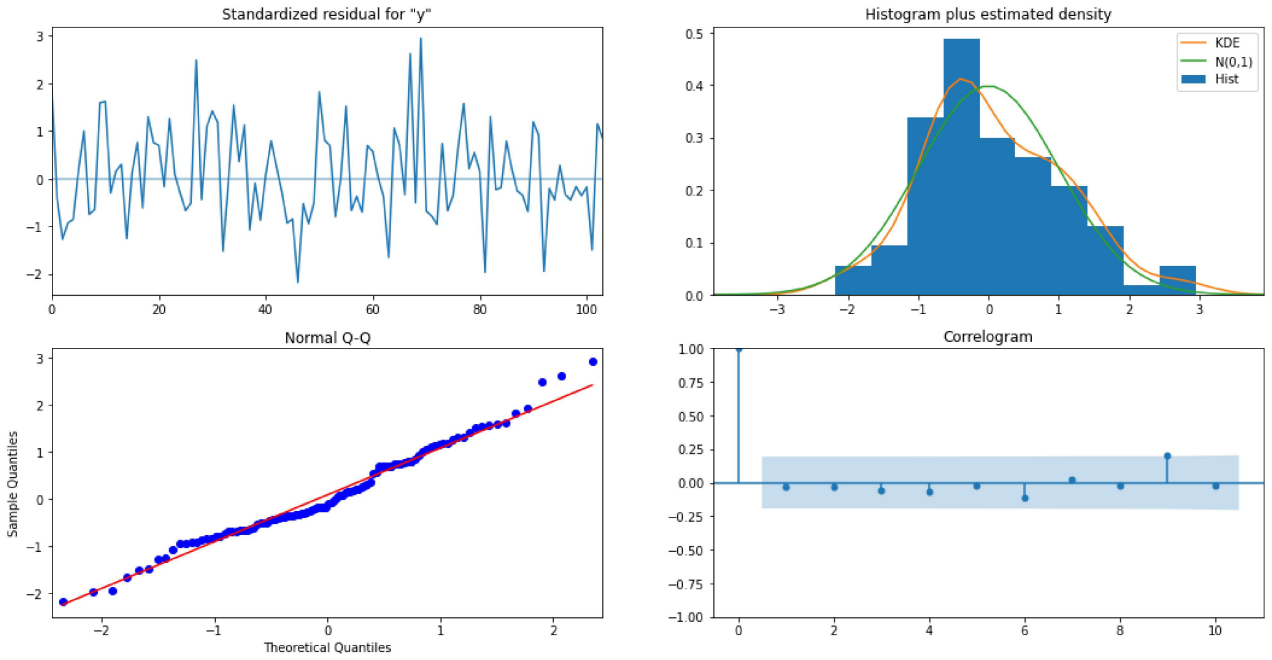
```
import statsmodels.api as sm

auto_SARIMA_12 = sm.tsa.statespace.SARIMAX(train['Rose'].values,
                                order=(0, 1, 2),
                                seasonal_order=(2, 0, 2, 12),
                                enforce_stationarity=False,
                                enforce_invertibility=False)
results_auto_SARIMA_12 = auto_SARIMA_12.fit(maxiter=1000)
print(results_auto_SARIMA_12.summary())
```

```
                                     SARIMAX Results
==========================================================================================
==
Dep. Variable:                                   y   No. Observations:                1
32
Model:             SARIMAX(0, 1, 2)x(2, 0, 2, 12)   Log Likelihood                -436.9
69
Date:                            Mon, 11 Oct 2021   AIC                             887.9
38
Time:                                    23:16:54   BIC                             906.4
48
Sample:                                         0   HQIC                            895.4
37
```

```
                                          - 132
Covariance Type:                            opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ma.L1         -0.8427    189.705     -0.004      0.996    -372.658     370.972
ma.L2         -0.1573     29.804     -0.005      0.996     -58.571      58.257
ar.S.L12       0.3467      0.079      4.375      0.000       0.191       0.502
ar.S.L24       0.3023      0.076      3.996      0.000       0.154       0.451
ma.S.L12       0.0767      0.133      0.577      0.564      -0.184       0.337
ma.S.L24      -0.0726      0.146     -0.498      0.618      -0.358       0.213
sigma2       251.3136   4.77e+04      0.005      0.996   -9.32e+04    9.37e+04
===================================================================================
Ljung-Box (L1) (Q):                   0.10   Jarque-Bera (JB):                 2.33
Prob(Q):                              0.75   Prob(JB):                         0.31
Heteroskedasticity (H):               0.88   Skew:                             0.37
Prob(H) (two-sided):                  0.70   Kurtosis:                         3.03
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

In [82]:

```
#Plotting

results_auto_SARIMA_12.plot_diagnostics()
plt.show()
```



In [83]:

```
#prediction on test set

predicted_auto_SARIMA_12 = results_auto_SARIMA_12.get_forecast(steps=len(test))
predicted_auto_SARIMA_12.summary_frame(alpha=0.05).head() #for alpha = 0.05
```

Out[83]:

	y	mean	mean_se	mean_ci_lower	mean_ci_upper
0		62.867263	15.928500	31.647976	94.086550

y	mean	mean_se	mean_ci_lower	mean_ci_upper
1	70.541190	16.147658	38.892361	102.190019
2	77.356410	16.147656	45.707586	109.005235
3	76.208814	16.147656	44.559989	107.857638
4	72.747398	16.147656	41.098573	104.396222

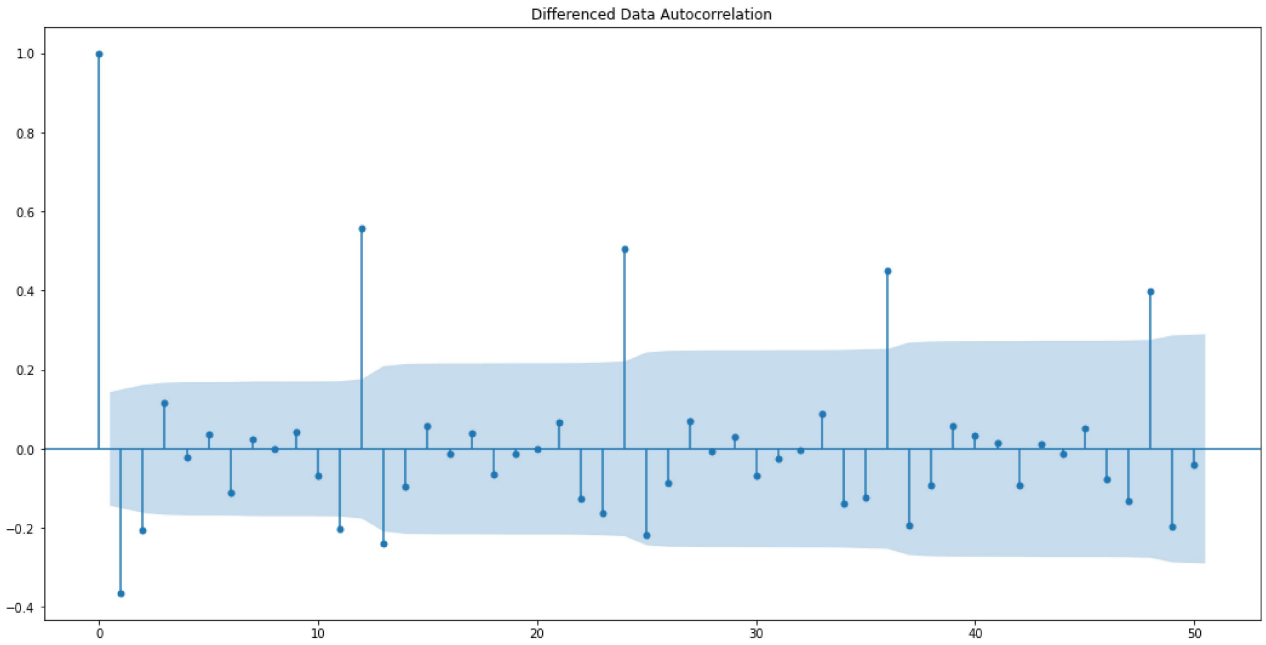
In [84]:
```
#Calculating RMSE on test data

rmse_2 = mean_squared_error(test['Rose'],predicted_auto_SARIMA_12.predicted_mean,square

print(rmse_2)
```

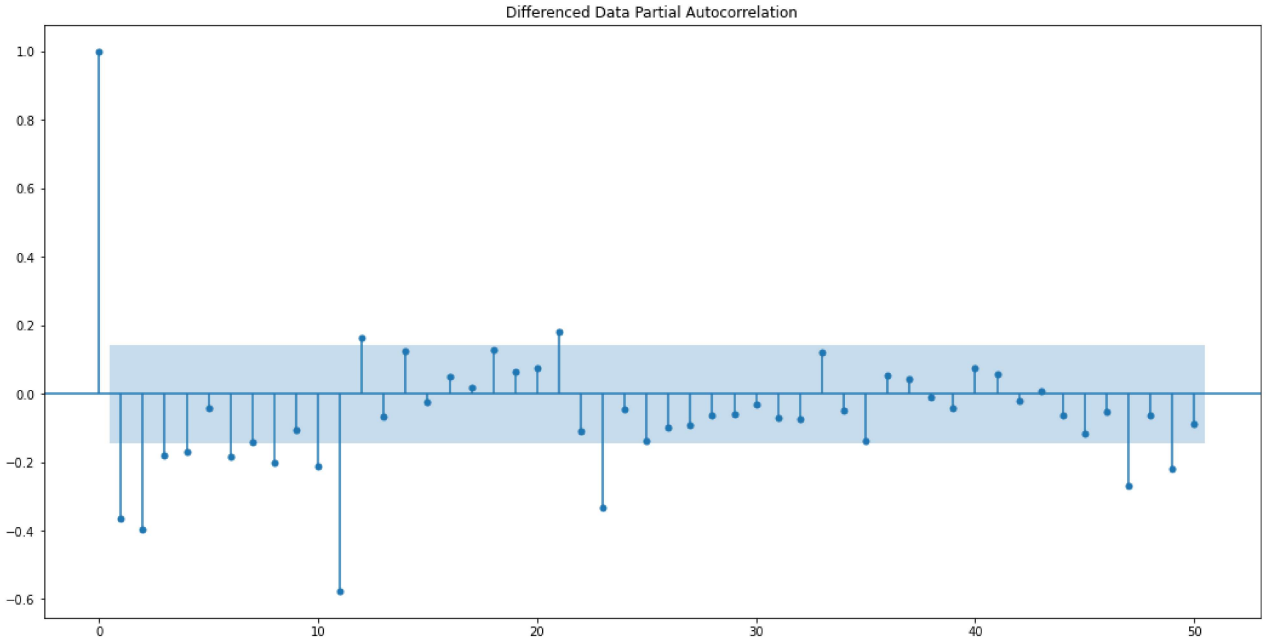
```
26.88062501761389
```

7. Build ARIMA/SARIMA models based on the cut-off points of ACF and PACF on the training data and evaluate this model on the test data using RMSE.

In [85]:
```
#Let's plot the ARIMA/SARIMA models with cut off points of ACF and PACF on training dat

#import plot_acf and plot_pacf

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

In [86]:
```
plot_acf(rose_data['Rose'].diff().dropna(),lags=50,title='Differenced Data Autocorrelat
plot_pacf(rose_data['Rose'].diff().dropna(),lags=50,title='Differenced Data Partial Aut
plt.show()
```





In [87]:

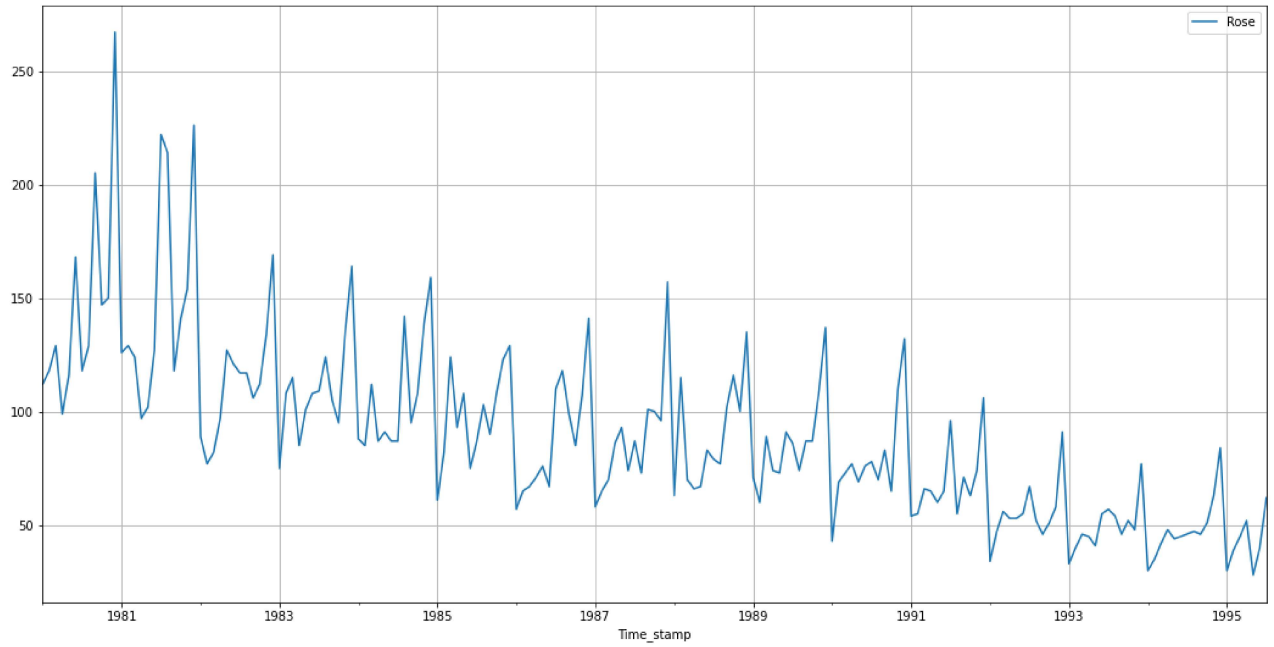
```
#PACF cut off is at lag 4 and
#and ACF plot cuts-off is at lag 2.

#Let's look at the summary points on this basis:

manual_ARIMA = ARIMA(train['Rose'].astype('float64'), order=(4,1,2),freq='M')
results_manual_ARIMA = manual_ARIMA.fit()
print(results_manual_ARIMA.summary())
```

```
                             ARIMA Model Results
==============================================================================
Dep. Variable:                 D.Rose   No. Observations:                  131
Model:                 ARIMA(4, 1, 2)   Log Likelihood                -633.876
Method:                       css-mle   S.D. of innovations             29.793
Date:                Mon, 11 Oct 2021   AIC                           1283.753
Time:                        23:16:56   BIC                           1306.754
Sample:                    02-29-1980   HQIC                          1293.099
                         - 12-31-1990
==================================================================================
                     coef    std err          z      P>|z|      [0.025      0.975]
----------------------------------------------------------------------------------
const             -0.1905      0.576     -0.331      0.741      -1.319       0.938
ar.L1.D.Rose       1.1685      0.087     13.391      0.000       0.997       1.340
ar.L2.D.Rose      -0.3562      0.132     -2.692      0.007      -0.616      -0.097
ar.L3.D.Rose       0.1855      0.132      1.402      0.161      -0.074       0.445
ar.L4.D.Rose      -0.2227      0.091     -2.443      0.015      -0.401      -0.044
ma.L1.D.Rose      -1.9506        nan        nan        nan         nan         nan
ma.L2.D.Rose       1.0000        nan        nan        nan         nan         nan
                                    Roots
=============================================================================
                  Real          Imaginary           Modulus         Frequency
-----------------------------------------------------------------------------
AR.1            1.1027           -0.4115j            1.1770           -0.0569
AR.2            1.1027           +0.4115j            1.1770            0.0569
AR.3           -0.6863           -1.6644j            1.8003           -0.3122
AR.4           -0.6863           +1.6644j            1.8003            0.3122
MA.1            0.9753           -0.2209j            1.0000           -0.0355
MA.2            0.9753           +0.2209j            1.0000            0.0355
-----------------------------------------------------------------------------
```

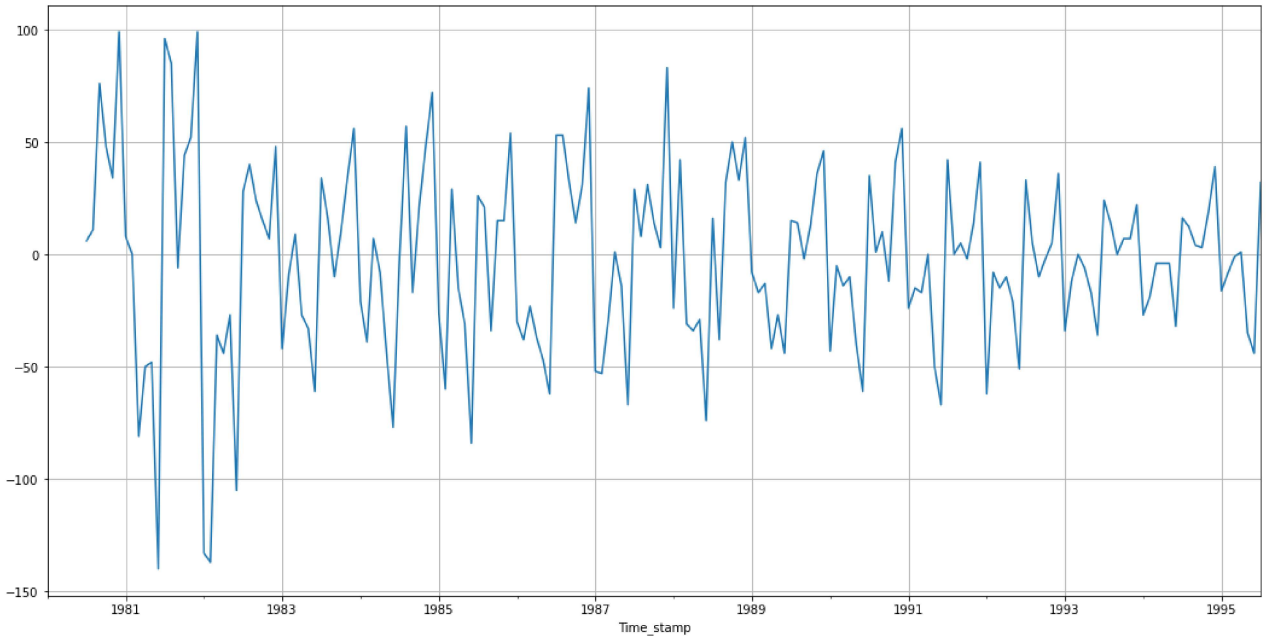
In [88]:
```
predicted_manual_ARIMA = results_manual_ARIMA.forecast(steps=len(test))

#RMSE

rmse_3 = mean_squared_error(test['Rose'],predicted_manual_ARIMA[0],squared=False)
print(rmse_3)
```

```
33.90482729155276
```

In [89]:
```
rose_data.plot()
plt.grid()
plt.show()
```

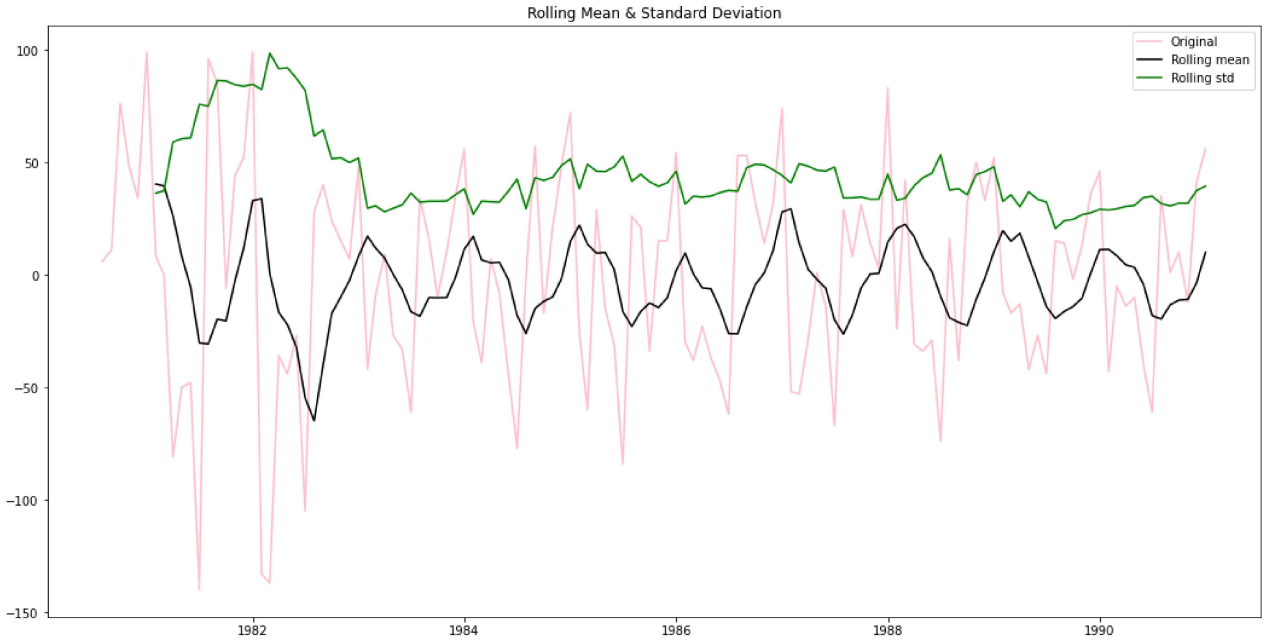


In [90]:
```
#checking it after plotting with the differnecing of 6
(rose_data['Rose'].diff(6)).plot()
plt.grid()
plt.show()
```



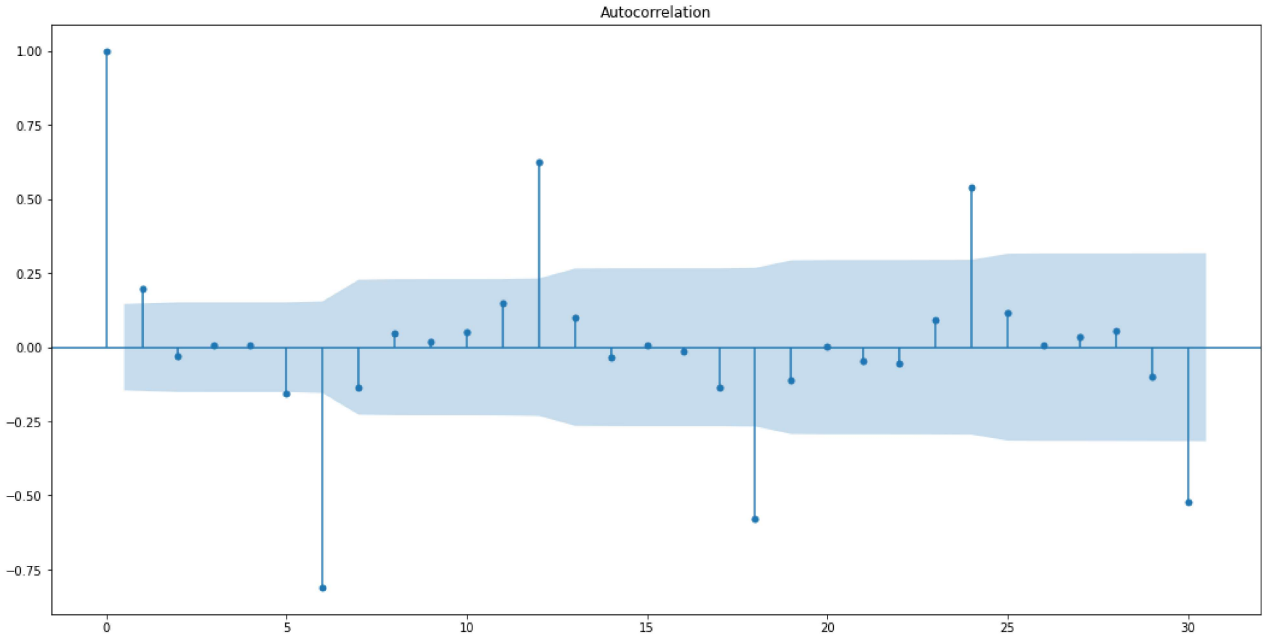
In [91]:

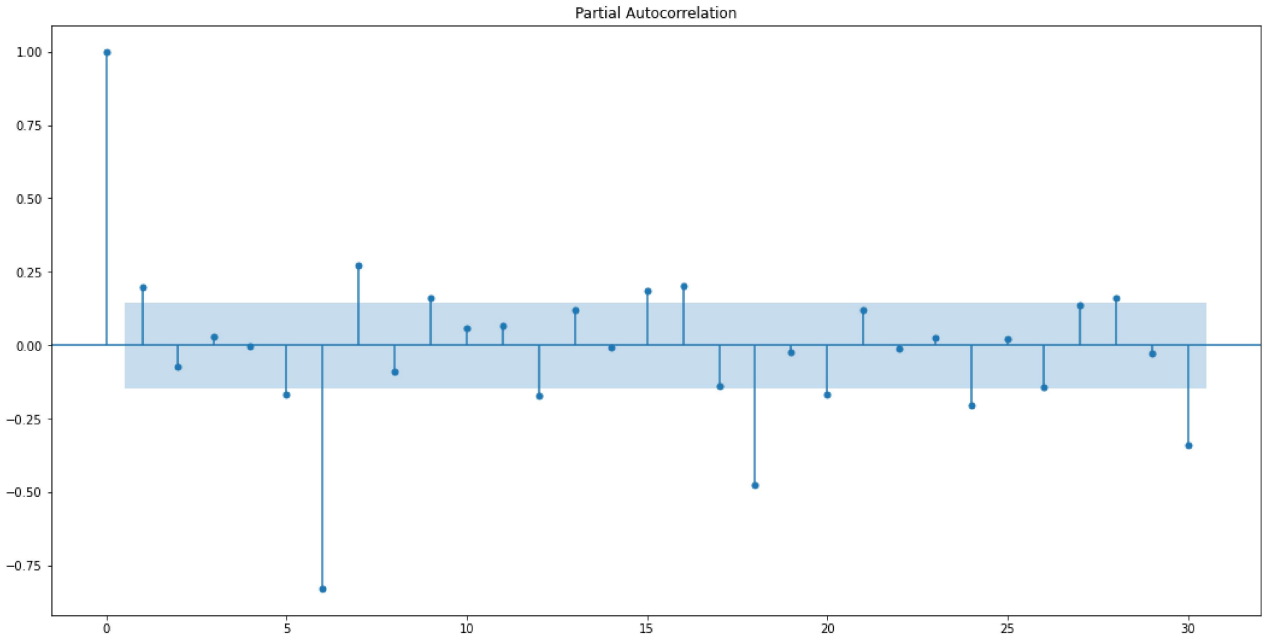
```
#testing stationarity at diff 6

Stationarity_test(train['Rose'].diff(6).dropna())
```



In [92]:

```
#again plotting acf and pacf with diff 6

plot_acf((rose_data['Rose'].diff(6).dropna()),lags=30)
plot_pacf((rose_data['Rose'].diff(6).dropna()),lags=30);
```





Now, both the ACF and PACF plots have the cut off at 1. Alpha value at 0.05. we will check the ACF and PACF plots at a multiple of 6, because seasonal period is 6

In [93]:
```
manual_SARIMA_model = sm.tsa.statespace.SARIMAX(train['Rose'].values,
                                order=(4, 1, 2),
                                seasonal_order=(1, 1, 1, 6),
                                enforce_stationarity=False,
                                enforce_invertibility=False)
results_manual_SARIMA_model = manual_SARIMA_model.fit(maxiter=1000)
```

In [94]:
```
print(results_manual_SARIMA_model.summary())
```

```
                                 SARIMAX Results
==========================================================================================
===
Dep. Variable:                                   y   No. Observations:
132
```

```
Model:             SARIMAX(4, 1, 2)x(1, 1, [1], 6)   Log Likelihood               -526.
344
Date:                             Mon, 11 Oct 2021   AIC                          1070.
688
Time:                                     23:17:00   BIC                          1095.
393
Sample:                                          0   HQIC                         1080.
716
                                               - 132
Covariance Type:                               opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.0722      0.270     -0.267      0.789      -0.601       0.457
ar.L2         -0.4955      0.141     -3.515      0.000      -0.772      -0.219
ar.L3         -0.0486      0.121     -0.401      0.688      -0.286       0.189
ar.L4         -0.3135      0.107     -2.926      0.003      -0.524      -0.104
ma.L1         87.5064   3459.696      0.025      0.980   -6693.373    6868.386
ma.L2        -56.4405   2197.671     -0.026      0.980   -4363.797    4250.916
ar.S.L6       -0.7330      0.066    -11.163      0.000      -0.862      -0.604
ma.S.L6       -1.4121      0.208     -6.779      0.000      -1.820      -1.004
sigma2         0.0341      2.673      0.013      0.990      -5.206       5.274
===================================================================================
Ljung-Box (L1) (Q):                   0.07   Jarque-Bera (JB):                 23.99
Prob(Q):                              0.79   Prob(JB):                          0.00
Heteroskedasticity (H):               0.36   Skew:                              0.44
Prob(H) (two-sided):                  0.00   Kurtosis:                          5.06
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

In [95]:
```
predicted_manual_SARIMA_model = results_manual_SARIMA_model.get_forecast(steps=len(test
```

In [96]:
```
#at aplha 0.05

predicted_manual_SARIMA_model.summary_frame(alpha=0.05).head()
```

Out[96]:

y	mean	mean_se	mean_ci_lower	mean_ci_upper
0	43.681048	22.984050	-1.366863	88.728958
1	66.918041	23.988326	19.901786	113.934296
2	71.109578	24.245548	23.589177	118.629979
3	78.880894	24.588962	30.687415	127.074373
4	67.838140	24.693580	19.439612	116.236668

In [97]:
```
#RMSE on test data

rmse_4 = mean_squared_error(test['Rose'],predicted_manual_SARIMA_model.predicted_mean,s

print(rmse_4)
```

```
19.004805582081588
```

8. Build a table with all the models built along with their corresponding parameters and the respective RMSE values on the test data.

In [98]:
```
#A table with all the RMSE values along with all the corresponding parameters :on test

#values sorted in ascending order

data = [['RegressionOnTime', RMSE_LR_test],
                ['NaiveModel', RMSE_NM_test],
                ['SimpleAverageModel', RMSE_SA_test],
                ['2pointTrailingMovingAverage', RMSE_MA_test_2 ],
                ['4pointTrailingMovingAverage', RMSE_MA_test_4],
                ['6pointTrailingMovingAverage', RMSE_MA_test_6 ],
                ['9pointTrailingMovingAverage', RMSE_MA_test_9],
       ['Alpha=0.065,Beta=0.051,Gamma=0.0004,TripleExponentialSmoothing',RMSE_TES_test]
       ['Alpha=0.3,Beta=0.4,Gamma=0.3,TripleExponentialSmoothing', RMSE_TES_test_new_1]
       ['ARIMA(0,1,2)',RMSE_1],['SARIMA(0,1,2)(2,0,2,12)',rmse_2],['ARIMA (4,1,2) based
        ['SARIMA(4,1,2)(1,1,1,6) based on ACF & PACF',rmse_4]]
```

In [99]:
```
ALL_RMSE = pd.DataFrame(data, columns = ['Model_Name', 'RMSE_Values'])
```

In [100…
```
print('Sorted by RMSE values on the Test Data:','\n',)

ALL_RMSE.sort_values(by=['RMSE_Values'])
```

```
Sorted by RMSE values on the Test Data:
```

Out[100…

	Model_Name	RMSE_Values
8	Alpha=0.3,Beta=0.4,Gamma=0.3,TripleExponential...	10.935749
3	2pointTrailingMovingAverage	11.529994
4	4pointTrailingMovingAverage	14.444342
5	6pointTrailingMovingAverage	14.554944
6	9pointTrailingMovingAverage	14.721499
0	RegressionOnTime	15.255435
9	ARIMA(0,1,2)	15.604196
12	SARIMA(4,1,2)(1,1,1,6) based on ACF & PACF	19.004806
7	Alpha=0.065,Beta=0.051,Gamma=0.0004,TripleExpo...	20.959970
10	SARIMA(0,1,2)(2,0,2,12)	26.880625
11	ARIMA (4,1,2) based on ACF & PACF	33.904827
2	SimpleAverageModel	53.413057
1	NaiveModel	79.672238

we can see that the Triple Exponential Smoothing with Alpha=0.3,Beta=0.4,Gamma=0.3,gives us the lowest RMSE value of 10.935749. So, this is the best model among all.

9. Based on the model-building exercise, build the most optimum model(s) on the complete data and predict 12 months into the future with appropriate confidence intervals/bands.

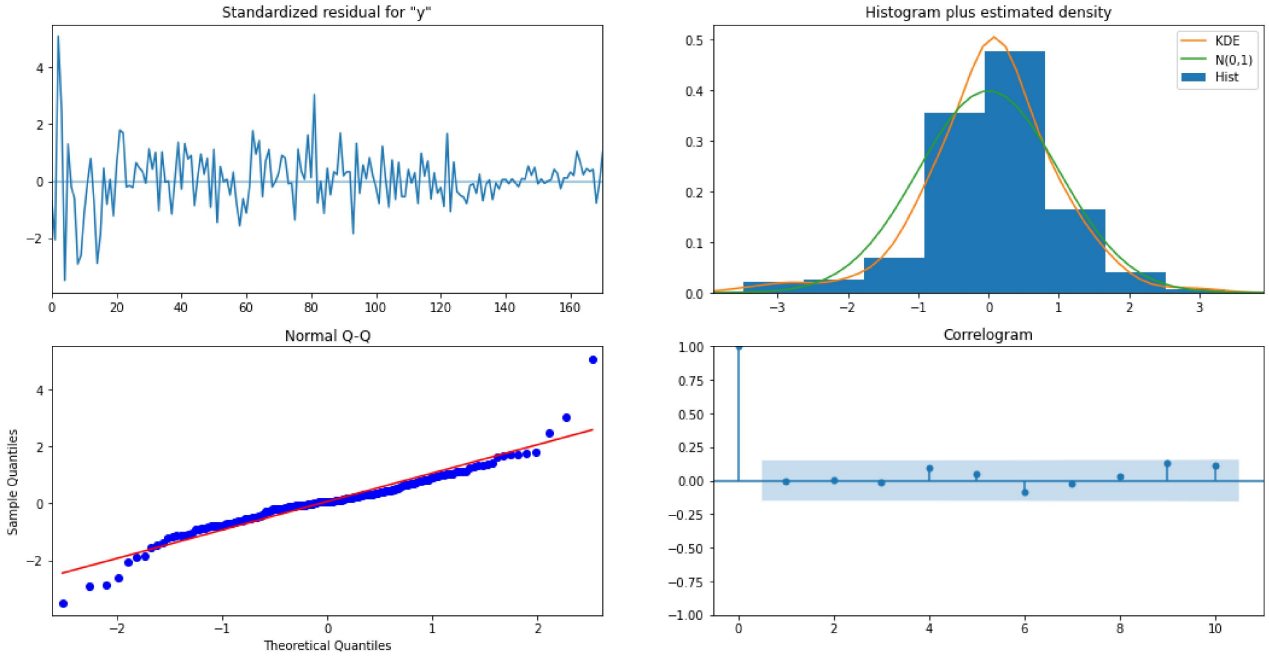
In [103…

```
optimum_model = sm.tsa.statespace.SARIMAX(rose_data['Rose'].values,
                                  order=(3,1,2),
                                  seasonal_order=(1,1,1,6),
                                  enforce_stationarity=False,
                                  enforce_invertibility=False)
results_optimum_model = optimum_model.fit(maxiter=1000)
print(results_optimum_model.summary())
```

```
                                     SARIMAX Results
==========================================================================================
===
Dep. Variable:                                     y   No. Observations:
187
Model:             SARIMAX(3, 1, 2)x(1, 1, [1], 6)   Log Likelihood                -763.
100
Date:                             Tue, 12 Oct 2021   AIC                           1542.
199
Time:                                     11:56:16   BIC                           1567.
332
Sample:                                          0   HQIC                          1552.
397
                                               - 187
Covariance Type:                                 opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.6994      0.049    -14.191      0.000      -0.796      -0.603
ar.L2         -0.0067      0.064     -0.105      0.916      -0.132       0.119
ar.L3         -0.1250      0.078     -1.609      0.108      -0.277       0.027
ma.L1         -0.0016      6.464     -0.000      1.000     -12.670      12.667
ma.L2         -1.0017      6.433     -0.156      0.876     -13.611      11.607
ar.S.L6       -0.8419      0.039    -21.520      0.000      -0.919      -0.765
ma.S.L6        0.1158      0.070      1.648      0.099      -0.022       0.254
sigma2       420.6666   2703.053      0.156      0.876   -4877.220    5718.554
===================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):               148.79
Prob(Q):                              0.98   Prob(JB):                         0.00
Heteroskedasticity (H):               0.14   Skew:                             0.28
Prob(H) (two-sided):                  0.00   Kurtosis:                         7.54
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

In [104…

```
#Diagnostics

results_optimum_model.plot_diagnostics();
```



In [105…
```
#evaluation of the model on full dataset

#as given in the question, we have to predict 12 months in the future

predicted_manual_SARIMA_model_optimum = results_optimum_model.get_forecast(steps=12)
```

In [106…
```
predicted_manual_SARIMA_model_optimum.summary_frame(alpha=0.05).head()
```

Out[106…

y	mean	mean_se	mean_ci_lower	mean_ci_upper
0	46.535356	20.646526	6.068910	87.001803
1	36.939114	21.558698	-5.315158	79.193386
2	45.260561	21.971991	2.196250	88.324872
3	49.103727	21.979033	6.025614	92.181840
4	70.531779	21.998015	27.416461	113.647097

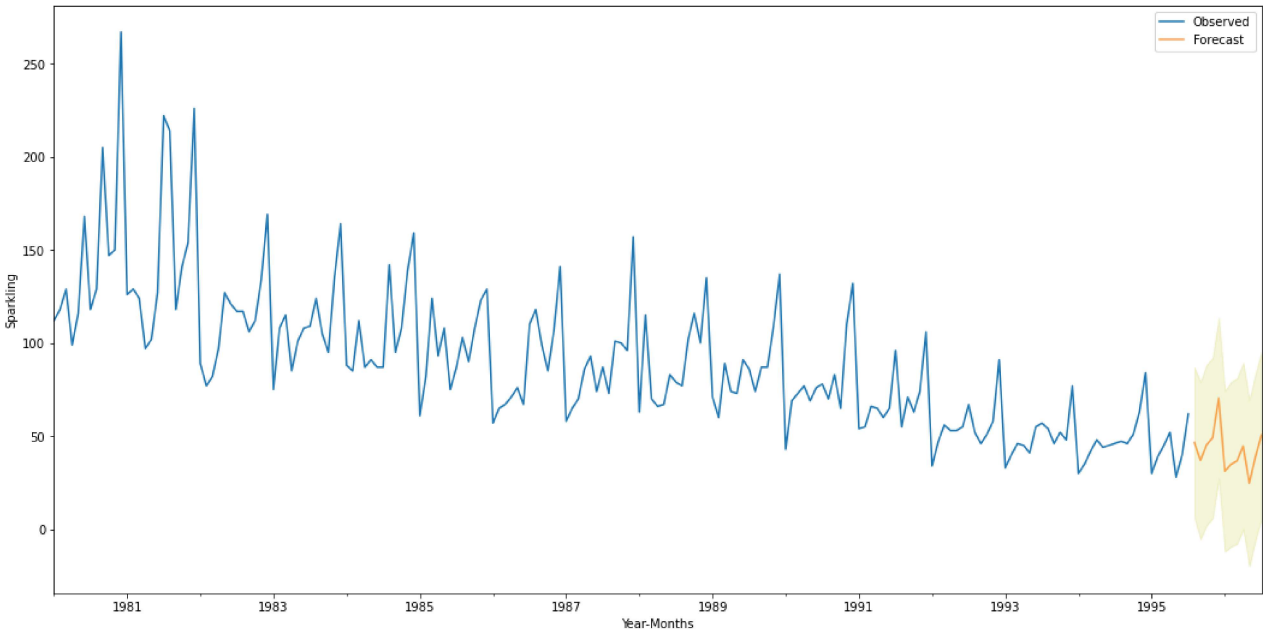
In [107…
```
#RMSE

rmse = mean_squared_error(rose_data['Rose'],results_optimum_model.fittedvalues,squared=
print('RMSE of the best optimum model on whole dataset is:',rmse)
```

```
RMSE of the best optimum model on whole dataset is: 28.82521013783407
```

In [108…
```
pred_full_manual_SARIMA_date = predicted_manual_SARIMA_model_optimum.summary_frame(alph
```

In [110…
```
#plotting

axis = rose_data['Rose'].plot(label='Observed')
pred_full_manual_SARIMA_date['mean'].plot(ax=axis, label='Forecast', alpha=0.7)
axis.fill_between(pred_full_manual_SARIMA_date.index, pred_full_manual_SARIMA_date['mea
```

```
                    pred_full_manual_SARIMA_date['mean_ci_upper'], color='y', alpha=.15)
axis.set_xlabel('Year-Months')
axis.set_ylabel('Sparkling')
plt.legend(loc='best')
plt.show()
```



10.Comment on the model thus built and report your findings and suggest the measures that the company should be taking for future sales.

Some Measures that company need to take care of in future:

We have observed the sales pattern of rose wine from the year 1980 till 1995 and found out that few points that need to understand:

Fall in trend is clearly visible from yearly plot.

The maximum number of sales happened in the month of December due to some festive season and also because of the year end celebration, so we need to take care of all the arrangements for the customers by keep our stocks full at all point of sales center/shop.

We also need to take care of quality, the purity of material used to make the rose wine. This will definitely lead to advancement in sales

The lowest number of sales observed in the month of June.

We have build all the applicable models with our data and calculated RMSE values on all the test data.

After calculating RMSE, we got to know the Triple Exponential model is best for our data due to it's lowest RMSE value.

We have also predicted the sales for upcoming 12 months with confidence interval.

Trend was affecting too much and not up to the mark as expected. So we also need to take care of the better quality of the wine to get the attention and attraction of the customers.

If we are able to take these above steps carefully then definitely our sales of the Rose wine would be more in upcoming future.

__THE END__